ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ
TEKNOLOJİ FAKÜLTESİ
ELEKTRİK-ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

İLERİ DÜZEY PROGRAMLAMA

C# WINDOWS FORMS İLE SCADA ARAYÜZÜ TASARIMI

Ceyhan – Kırıkkale Ham Petrol Boru Hattı İzleme ve Sızıntı Tespit Sistemi

Ara Sınav Proje Raporu

Hazırlayan
Yankı ÇİLİNGİR
Öğrenci No: 2112705020

İÇİNDEKİLER

1. PROJENİN TANITIMI 4
1.1. SCADA Nedir? 4
1.2. Projenin Amacı 4
1.3. Kurgulanan Sistem Senaryosu 4
1.4. Sistemde Bulunan Cihazlar 5
1.5. Ölçülen (Simüle Edilen) Büyüklükler 5
1.6. Kullanıcının Kontrol Edebildiği Elemanlar 5
1.7. Programın Genel Olarak Yaptığı İş 5
2. PROGRAMIN GENEL ÇALIŞMA YAPISI 6
2.1. Program Başladığında Yapılanlar 6
2.2. Her Saniye Yapılan İşlemler 6
2.3. Bileşenler Arasındaki İlişki 6
2.4. Sensör Değerlerinin Üretilmesi 7
2.5. Alarm Koşullarının Çalışması 7
3. KULLANILAN BİLEŞENLERİN EKLENMESİ 7
3.1. Projenin Oluşturulması 7
3.2. ProgressBar Eklenmesi 8
3.3. NumericUpDown Eklenmesi 9
3.4. Timer Eklenmesi 9
3.5. Button Eklenmesi ve Click Olayının Oluşturulması 10
3.6. Diğer Bileşenler 11
4. ARAYÜZ TASARIMININ OLUŞTURULMASI 11
4.1. Proses Elemanlarının Ekranda Gösterilmesi 11
4.2. Geliştirme Aşamaları 12
5. KODLAMA 14
5.1. Değişkenlerin Tanımlanması 14
5.2. Formun Yüklenmesi 15
5.3. Simülasyon Timer'ının Tick Olayı 15
5.4. Debinin Hesaplanması ve Random ile Sensör Verisi Üretilmesi 16
5.5. Basıncın Hesaplanması 17
5.6. Tank Seviyesinin Hesaplanması 18
5.7. Sızıntı Tespiti 19
5.8. Ekranın Güncellenmesi ve Renklerin Değiştirilmesi 20
5.9. Kullanıcıdan Veri Alma ve Hata Kontrolü 21
6. TIMER VE VERİ SİMÜLASYONU 22
6.1. Timer Neden Kullanıldı? 22

6.2. Interval Değerleri ve Tick Olayları 22
6.3. Değerlerin Sistem Durumuna Göre Değişmesi 23
7. SİSTEM KONTROLLERİNİN AÇIKLANMASI 24
7.1. Hattı Başlatma ve Durdurma 24
7.2. Vana Açma ve Kapatma 25
7.3. Sızıntılı Bölümün İzole Edilmesi 25
7.4. Arıza Simülasyonu Butonları 26
8. ALARM MEKANİZMASI 26
8.1. Alarm Koşulları ve Sınırları 26
8.2. Alarmların Kontrol Edilmesi 27
8.3. Alarm Sırasında Arayüzdeki Değişiklikler 28
9. PROGRAMIN ÇALIŞTIRILMASI VE TEST EDİLMESİ 29
9.1. İlk Açılış ve Normal Çalışma 29
9.2. Sızıntı Alarmı, Alarm Onayı ve İzolasyon 30
9.3. Haberleşme Kaybı ve Alarmın Ortadan Kalkması 31
9.4. Vana Kapatılması ve Yüksek Basınç 32
9.5. Pompa Arızası 33
9.6. Hatalı Kullanıcı Girişi 33
9.7. Rastgele Arıza 34
9.8. Tank Seviyesi Alarmları 34
10. PROGRAMIN SON HÂLİ 36
11. SONUÇ 36
11.1. Gerçekleştirilenler 36
11.2. Kullanılan C# Konuları 37
11.3. Karşılaşılan Problemler ve Çözümleri 37
11.4. Kazanımlar 38

1. PROJENİN TANITIMI

1.1. SCADA Nedir?

SCADA (Supervisory Control and Data Acquisition – Gözetleyici Kontrol ve Veri Toplama), sahaya dağılmış cihazlardan ölçüm toplayan, bu ölçümleri operatöre bir ekran üzerinden gösteren ve operatörün sahadaki pompa, vana gibi ekipmanlara uzaktan komut göndermesini sağlayan sistemlerin genel adıdır. Gerçek bir SCADA sisteminde veriler sahadaki PLC ve RTU (uzak terminal ünitesi) cihazlarından haberleşme hatları üzerinden kontrol merkezine gelir. Operatörün baktığı ekrana HMI (Human Machine Interface – İnsan Makine Arayüzü) denir.

Yüzlerce kilometrelik boru hatları SCADA'nın tipik kullanım alanlarındandır, çünkü pompa ve vana istasyonları tek bir merkezden izlenir. Bu projede böyle bir boru hattı için operatör ekranı hazırlanmıştır.

1.2. Projenin Amacı

Projenin amacı, C# Windows Forms kullanarak bir ham petrol boru hattını izleyen ve kontrol eden SCADA benzeri bir operatör arayüzü geliştirmektir. Gerçek sensör kullanılmadığı için sensör değerleri programın içinde Timer ve Random sınıfları ile üretilmiştir. Değerler birbirinden bağımsız değildir: pompa hızı debiyi, debi basıncı ve tank seviyesini etkiler.

Projede Form, Label, Button, TextBox, NumericUpDown, ComboBox, ProgressBar, Panel, GroupBox, ListView ve Timer bileşenleri kullanılmıştır. Kodda değişkenler, diziler, if/else yapıları, döngüler, metotlar, olaylar (event), Random sınıfı, kullanıcıdan veri alma ve hata kontrolü bir arada kullanılmıştır.

1.3. Kurgulanan Sistem Senaryosu

Senaryo olarak Türkiye'deki gerçek **Ceyhan – Kırıkkale Ham Petrol Boru Hattı** örnek alınmıştır. Bu hat, Ceyhan'daki terminalden aldığı ham petrolü yaklaşık 448 km boyunca taşıyarak Kırıkkale Rafinerisi'ne ulaştırır. Hat 24 inç çapındadır ve saatte yaklaşık 1000–1140 m^3 petrol taşıyabilmektedir. Petrolün Toros Dağları'nı aşabilmesi için hat üzerinde iki pompa istasyonu bulunur: Ceyhan'daki çıkış pompası ve Pozantı'daki ara pompa istasyonu.

Projede hattın uzunluğu, istasyonların sırası ve normal debi değeri gerçek hattan alınmıştır. Basınç değerleri, alarm sınırları ve vanaların yerleri ise hattın hidrolik yapısına uygun olacak şekilde temsili olarak belirlenmiştir. Hat üzerinde altı vana istasyonu vardır: Ceyhan (km 0), Adana (km 60), Pozantı (km 112), Niğde (km 170), Aksaray (km 300) ve Kırıkkale (km 448). Hattın sonunda rafinerinin besleme tankı bulunur.

Boru hattında bir sızıntı olduğunda gerçek işletmelerde petrol başka bir hatta aktarılmaz. Tek hatlı sistemlerde uygulanan yöntem pompaları durdurmak ve sızıntılı bölümün iki yanındaki vanaları kapatarak o bölümü izole etmektir. Projedeki “Bölümü İzole Et” butonu bu işlemi yapar.

1.4. Sistemde Bulunan Cihazlar

Tablo 1. Sistemdeki cihazlar ve ekrandaki karşılıkları

Cihaz	Görevi	Ekranda gösterimi
P-101 Ceyhan pompası	Petrolü Ceyhan'dan hatta basar.	Label (yeşil: çalışıyor, gri: durdu)
P-201 Pozantı pompası	Toros geçişi için basıncı yeniden yükseltir.	Label (yeşil / gri / kırmızı: arızalı)
V1 – V6 vanaları	Hattı bölümlere ayırır, sızıntıda bölümü izole eder.	Label (yeşil: açık, beyaz: kapalı, gri: bağlantı yok)
Boru hattı	İstasyonları birbirine bağlar.	Panel (koyu gri, sızıntıda kırmızı)
Rafineri tankı	Kırıkkale'ye gelen petrolü depolar (1500 m^3).	Panel içinde dolan mavi Panel
İstasyon RTU'ları	Debi ve basınç ölçümünü merkeze gönderir.	İstasyon altındaki Label

1.5. Ölçülen (Simüle Edilen) Büyüklükler

Tablo 2. Simüle edilen ölçümler

Büyüklük	Birim	Aralık	Nerede gösteriliyor
Debi (her istasyonda)	m^3/h	0 – 1375	İstasyon etiketleri, Sızıntı Tespiti bölümü
Basınç (her istasyonda)	bar	5 – 84	İstasyon etiketleri
Tank seviyesi	%	0 – 100	Tank paneli ve yüzde etiketi
Giriş – çıkış debi farkı	%	0 – 10	ProgressBar ve “Fark” etiketi
Pompa hızı (set değeri)	%	0 – 100	NumericUpDown

1.6. Kullanıcının Kontrol Edebildiği Elemanlar

- **Hattı Başlat / Hattı Durdur:** İki pompayı birlikte çalıştırır veya durdurur.
- **Pompa hızı (%):** NumericUpDown ile pompa hızı girilir. Hız değişince debi ve basınç değişir.
- **Vana seçimi, Vanayı Aç / Vanayı Kapat:** Listeden seçilen vana açılır veya kapatılır.
- **Alarmları Onayla:** Yanıp sönen yeni alarmları “Görüldü” durumuna getirir.
- **Bölümü İzole Et:** Sızıntı tespit edildiğinde pompaları durdurur ve sızıntılı bölümün iki yanındaki vanaları kapatır.
- **Arıza Simülasyonu bölümü:** Seçilen bölümde sızıntı oluşturma, istasyonla haberleşmeyi kesme, P-201 pompa arızası, tank seviyesini ayarlama, rastgele arıza oluşturma ve arızaları giderme.

1.7. Programın Genel Olarak Yaptığı İş

Operatör hattı başlattığında debi birkaç saniyede yaklaşık 1100 m^3/h'e çıkar. Her saniye istasyonların debisi ve basıncı ile tank seviyesi hesaplanır ve alarmlar kontrol edilir. Program, Ceyhan'dan basılan ve Kırıkkale'ye ulaşan petrolü karşılaştırarak sızıntıyı tespit eder ve istasyon debilerinden sızıntının yerini bulur. Operatör durumu alarm listesinden görür ve butonlarla müdahale eder.

2. PROGRAMIN GENEL ÇALIŞMA YAPISI

2.1. Program Başladığında Yapılanlar

Program `Program.cs` dosyasındaki `Main` metodundan başlar ve `Application.Run(new AnaForm())` satırı ile ana form açılır. Formun kurucu metodunda `InitializeComponent()` çağrılır. Bu metot tasarım ekranında eklenen bütün bileşenleri oluşturur. Form ekranda ilk kez gösterilirken `AnaForm_Load` olayı çalışır ve şu işlemleri yapar:

1. Vana etiketleri, istasyon bilgi etiketleri ve boru panelleri dizilere konur. Böylece altı istasyon tek bir döngü ile güncellenebilir.
2. Açılır listelerde (ComboBox) ilk seçili satır ayarlanır.
3. Ana simülasyon Timer'ı (1000 ms) ve alarm yakıp söndürme Timer'ı (500 ms) başlatılır.
4. `EkraniGuncelle()` çağrılarak ekrandaki bütün yazılar ilk değerleriyle doldurulur.

2.2. Her Saniye Yapılan İşlemler

Programın kalbi `tmrSimulasyon` Timer'ıdır. Interval değeri 1000 ms olduğu için Tick olayı saniyede bir çalışır ve aşağıdaki metotları sırayla çağırır. Sıra önemlidir, çünkü her adım bir önceki adımın sonucunu kullanır:

Tablo 3. Simülasyon Timer'ının her saniye çağırdığı metotlar

Sıra	Metot	Yaptığı iş	Kullandığı bilgi
1	DebiyiHesapla()	Hat debisini ve her istasyonun debisini hesaplar.	Pompa durumu, pompa hızı, vanalar, sızıntı
2	BasinciHesapla()	Her istasyonun basıncını hesaplar.	1. adımdaki debi, kapalı vana
3	TankiHesapla()	Tank seviyesini günceller.	Kırıkkale debisi
4	SizintiKontrolEt()	Giriş-çıkış farkına bakarak sızıntı arar.	Ceyhan ve Kırıkkale debisi
5	AlarmlariKontrolEt()	Alarm listesine alarm ekler veya siler.	Bütün durum değişkenleri
6	EkraniGuncelle()	Bütün sonuçları ekrana yazar, renkleri değiştirir.	Bütün değişkenler

2.3. Bileşenler Arasındaki İlişki

Ekrandaki bileşenler birbirine değişkenler üzerinden bağlıdır. Operatör bir butona bastığında buton doğrudan ekranı değiştirmez. Bunun yerine bir değişkeni değiştirir (örneğin `p101Calisiyor = true`). Bir sonraki Timer adımında hesaplama metotları bu değişkene göre yeni debi ve basınç değerlerini bulur, `EkraniGuncelle()` metodu da bu değerleri Label, Panel ve ProgressBar bileşenlerine yazar. Bu yapı sayesinde hesaplama ile ekran gösterimi birbirinden ayrılmış olur.

Örnek bir zincir şöyledir: Operatör pompa hızını %80'den %100'e çıkarır → `DebiyiHesapla()` hedef debiyi 1375 m³/h yapar → debi her saniye artar → `BasinciHesapla()` artan debiye göre pompa çıkış basıncını yükseltir → `TankiHesapla()` tanka gelen petrol arttığı için seviyeyi yükseltir → seviye %85'i geçerse `AlarmlariKontrolEt()` "Tank seviyesi yüksek" alarmını verir → `EkraniGuncelle()` tank panelini büyütür ve alarm bandını turuncu yapar.

2.4. Sensör Değerlerinin Üretilmesi

Debi, pompa hızından hesaplanan değere Random ile küçük bir sensör hatası eklenerek üretilir (5.4). Basınç debiden, tank seviyesi Kırıkkale'ye ulaşan debiden hesaplanır. Rastgele arızanın türü, yeri ve zamanı da Random ile seçilir (6.2).

2.5. Alarm Koşullarının Çalışması

Her saniye `AlarmlariKontrolEt()` metodu alarm koşullarına bakar. Koşul sağlanınca alarm listeye eklenir, ortadan kalkınca silinir. Yeni alarmlar onaylanana kadar yanıp söner (8. bölüm).

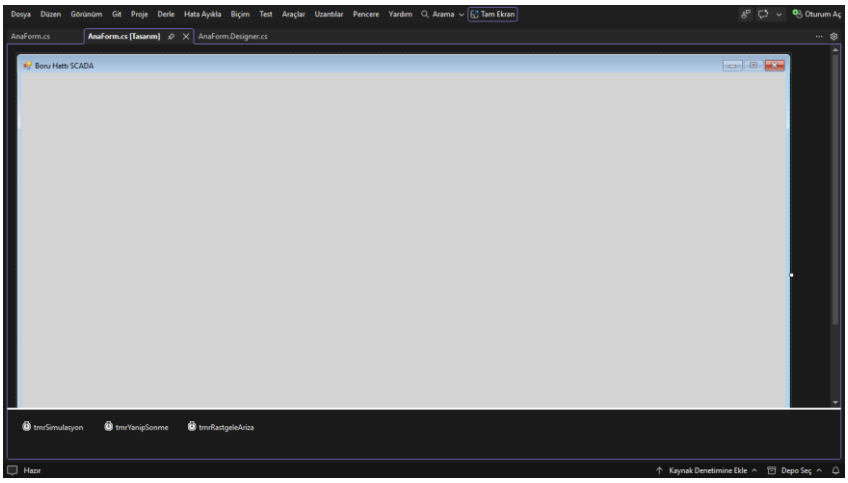
3. KULLANILAN BİLEŞENLERİN EKLENMESİ

3.1. Projenin Oluşturulması

1. Visual Studio açıldı, **Yeni proje oluştur** seçildi.
2. Şablon listesinden **Windows Forms Uygulaması (.NET Framework)** – C# seçildi.
3. Proje adı `BoruHattiScada` olarak girildi, .NET Framework 4.8 seçilerek proje oluşturuldu.
4. Çözüm Gezgini'nde `Form1.cs` dosyasına sağ tıklanıp **Yeniden Adlandır** ile adı `AnaForm.cs` yapıldı. Visual Studio, sınıf adını da AnaForm olarak değiştirdi.
5. Form seçilerek Özellikler (Properties) penceresinde aşağıdaki ayarlar yapıldı.

Tablo 4. Formun Properties ayarları

Özellik	Değer	Açıklama
Text	Boru Hattı SCADA	Pencerenin başlık yazısı
ClientSize	1240; 680	Formun iç boyutu (piksel)
FormBorderStyle	FixedSingle	Kullanıcı form boyutunu değiştiremez, tasarım bozulmaz
MaximizeBox	False	Ekranı kaplama butonu kapatıldı
StartPosition	CenterScreen	Form ekranın ortasında açılır
BackColor	212; 212; 212	SCADA ekranlarında kullanılan açık gri zemin
Font	Segoe UI; 9pt	Bütün bileşenlerde okunaklı yazı tipi

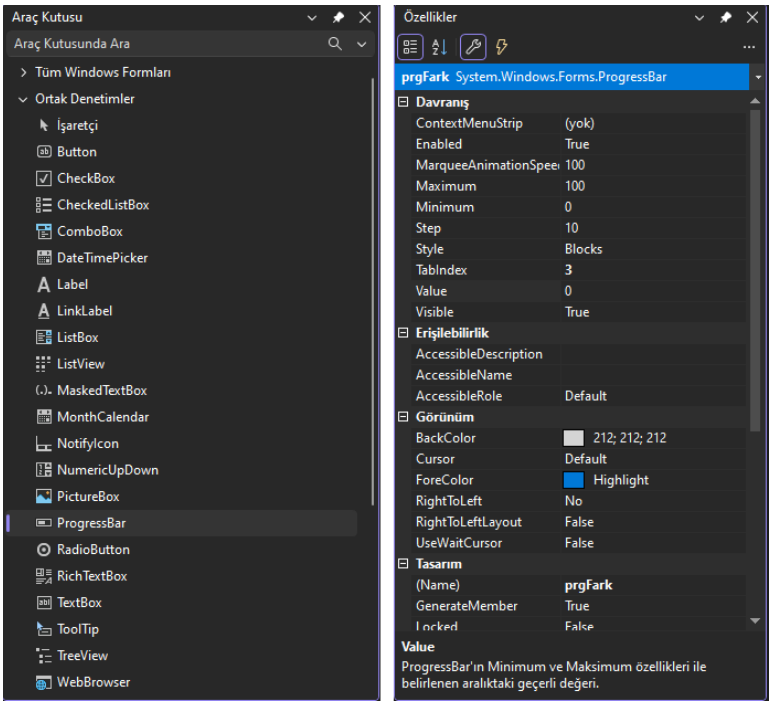


Şekil 1. Formun Properties ayarları yapıldıktan sonraki ilk hâli. Alt kısımdaki gri alanda formda görünmeyen Timer bileşenleri yer alır.

3.2. ProgressBar Eklenmesi

Ceyhan'dan basılan ile Kırıkkale'ye ulaşan petrol arasındaki farkı görsel olarak göstermek için bir ProgressBar kullanılmıştır. Ekleme adımları şöyledir:

1. **Görünüm → Araç Kutusu** menüsü ile Toolbox açıldı.
2. Toolbox → **Ortak Denetimler** (Common Controls) grubundan **ProgressBar** seçildi (Şekil 2, solda).
3. ProgressBar, "Sızıntı Tespiti" GroupBox'ının içine sürüklendi.
4. Properties penceresinde **(Name)** özelliği `prgFark` yapıldı. **Minimum** değeri 0, **Maximum** değeri 100 olarak bırakıldı (Şekil 2, sağda).



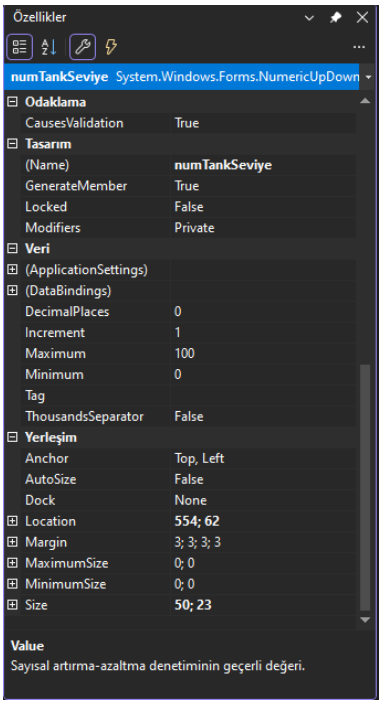
Şekil 2. Solda Toolbox penceresinde Ortak Denetimler grubundan ProgressBar'ın seçilmesi, sağda forma eklenen prgFark isimli ProgressBar'ın Properties penceresi (Minimum = 0, Maximum = 100, Value = 0).

Fark yüzdesi %0 ile %10 arasında olduğu için kodda fark 10 ile çarpılarak ProgressBar'a yazılır. Böylece %10 farkta çubuk tamamen dolar. Bu kod 5.8 bölümünde açıklanmıştır.

3.3. NumericUpDown Eklenmesi

Kullanıcının sayı girmesi gereken iki yerde NumericUpDown kullanılmıştır: pompa hızı (`numHiz`) ve tank seviyesi ayarı (`numTankSeviye`). NumericUpDown, Minimum ve Maximum değerleri dışında sayı girilmesine izin vermediği için ayrıca hata kontrolüne gerek kalmaz.

1. Toolbox → Ortak Denetimler → **NumericUpDown** seçilip "Arıza Simülasyonu" GroupBox'ına sürüklendi.
2. (Name) özelliği `numTankSeviye` yapıldı.
3. Minimum = 0, Maximum = 100, Value = 62 olarak ayarlandı (Şekil 3).
4. Pompa hızı için eklenen `numHiz` bileşeninde Increment = 5 ve Value = 80 yapıldı. Böylece oklara her basışta hız %5 değişir.



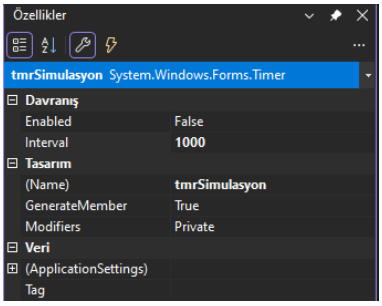
Şekil 3. numTankSeviye isimli NumericUpDown'ın Properties penceresi (Maximum = 100, Minimum = 0, Location ve Size).

3.4. Timer Eklenmesi

Timer, formun üzerinde görünmeyen bir bileşendir. Toolbox → **Bileşenler** (Components) grubundan **Timer** seçilip forma sürüklendiğinde formun altındaki gri alana (bileşen tepsisi) yerleşir. Projede üç Timer kullanılmıştır:

Tablo 5. Projede kullanılan Timer bileşenleri

Name	Interval	Görevi
tmrSimulasyon	1000 ms	Her saniye debi, basınç, tank ve alarm hesaplarını yapar.
tmrYanipSonme	500 ms	Yeni alarmları yarım saniyede bir yakıp söndürür.
tmrRastgeleAriza	Kodda 5000–20000 ms	Rastgele arızayı rastgele bir anda bir kez oluşturur.

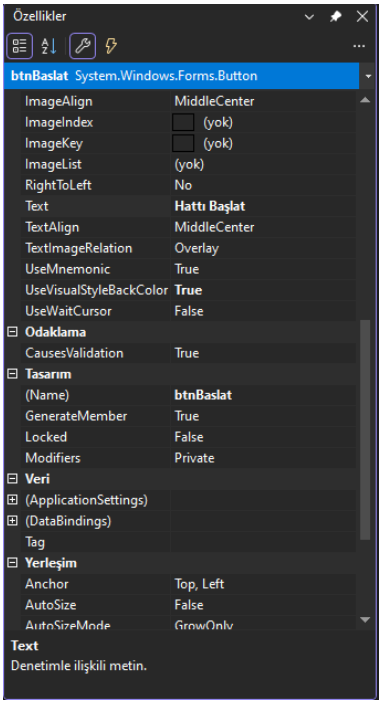


Şekil 4. tmrSimulasyon Timer'ının Properties penceresi (Interval = 1000, Enabled = False).

Timer'ın Tick olayını oluşturmak için bileşen tepsisindeki Timer'a çift tıklanmıştır. Visual Studio, `tmrSimulasyon_Tick` isimli boş bir metot oluşturur ve bu metodu Timer'ın Tick olayına bağlar. Enabled özelliği False bırakılmıştır, Timer'lar kod içinde `Start()` ile başlatılır.

3.5. Button Eklenmesi ve Click Olayının Oluşturulması

1. Toolbox → Ortak Denetimler → **Button** seçilip "Hat Kontrolü" GroupBox'ına sürüklendi.
2. (Name) özelliği `btnBaslat`, **Text** özelliği "Hattı Başlat" yapıldı (Şekil 5).
3. Tasarım ekranında butona çift tıklandı. Visual Studio `btnBaslat_Click` metodunu oluşturdu ve kod ekranını açtı.



Şekil 5. btnBaslat isimli butonun Properties penceresi ((Name) = btnBaslat, Text = Hattı Başlat).

Çift tıklama sonucunda Visual Studio, `AnaForm.Designer.cs` dosyasına butonu Click olayına bağlayan aşağıdaki satırı kendisi ekler. Aynı bağlantı Properties penceresindeki şimşek simgesi (Olaylar) ile de yapılabilir:

```
this.btnBaslat.Click += new System.EventHandler(this.btnBaslat_Click);
```

Diğer butonlar da aynı yöntemle eklenmiş ve her biri için Click metodu oluşturulmuştur.

3.6. Diğer Bileşenler

- **ComboBox:** Vana, istasyon ve sızıntı bölümü seçimi için kullanıldı. DropDownStyle = DropDownList yapılarak kullanıcının listede olmayan bir değer yazması engellendi. Seçenekler Items özelliğindeki listeye girildi.
- **TextBox:** Sızıntı miktarı (`txtSizintiMiktar`) kullanıcıdan TextBox ile alındı. Kullanıcı harf veya sınır dışı bir sayı yazabileceği için kodda hata kontrolü yapıldı (5.9 bölümü).
- **ListView:** Alarm listesi için kullanıldı. View = Details yapılıp Columns özelliğinden Zaman, Öncelik, Ekipman, Açıklama ve Durum sütunları eklendi.
- **GroupBox:** Ekran; Alarmlar, Hat Kontrolü, Sızıntı Tespiti ve Arıza Simülasyonu olmak üzere dört bölüme ayrıldı.
- **Panel ve Label:** Borular, tank, pompalar ve vanalar Panel ve Label bileşenleri ile çizildi (4. bölüm).

Tablo 6. Önemli bileşenlerin isimleri

Bileşen türü	Name	Görevi
Button	btnBaslat, btnDurdur	Hattı başlatma / durdurma
Button	btnVanaAc, btnVanaKapat	Seçilen vanayı açma / kapatma
Button	btnAlarmOnayla, btnIzoleEt	Alarm onaylama, sızıntılı bölümü izole etme
Button	btnSizintiOlustur, btnBaglantiKes, btnPompaArizasi, btnTankAyarla, btnRastgeleAriza, btnArizalariGider	Arıza simülasyonu
NumericUpDown	numHiz, numTankSeviye	Pompa hızı ve tank seviyesi girişi
TextBox	txtSizintiMiktar	Sızıntı miktarı girişi
ComboBox	cmbVana, cmbIstasyon, cmbSizintiBolum	Vana, istasyon ve bölüm seçimi
ProgressBar	prgFark	Giriş – çıkış farkı göstergesi
ListView	lvAlarmlar	Alarm listesi
Label	lblP101, lblP201, lblV1 … lblV6	Pompa ve vana sembolleri
Label	lblCeyhan … lblKirikkale	İstasyon basınç ve debi değerleri
Panel	pnlBoru1 … pnlBoru5, pnlTank, pnlTankDolu	Borular ve tank
Timer	tmrSimulasyon, tmrYanipSonme, tmrRastgeleAriza	Zamanlama

4. ARAYÜZ TASARIMININ OLUŞTURULMASI

4.1. Proses Elemanlarının Ekranda Gösterilmesi

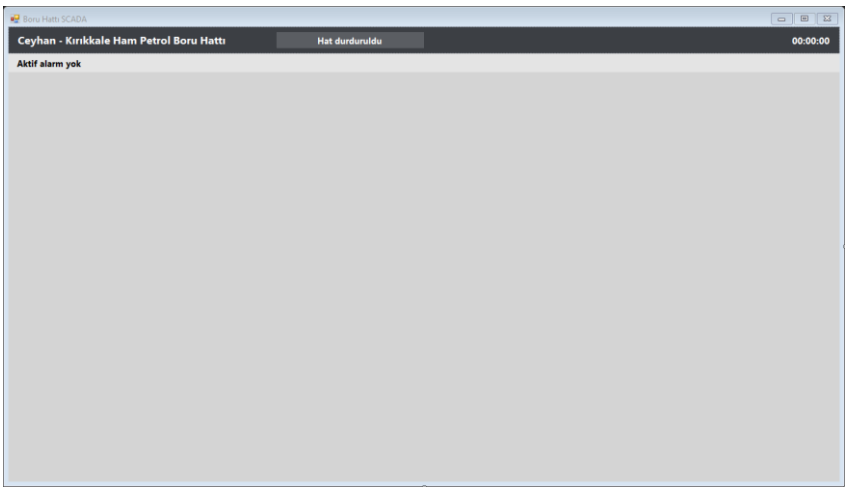
Hazır resim kullanılmamış, semboller Panel ve Label ile çizilmiştir. Böylece cihazın durumu yalnızca rengi değiştirilerek gösterilir.

- **Boru:** Yüksekliği 8 piksel olan uzun, koyu gri Panel. İki istasyon arasında bir boru parçası vardır, toplam 5 parça kullanılmıştır. Sızıntı olan parça kırmızıya döner.

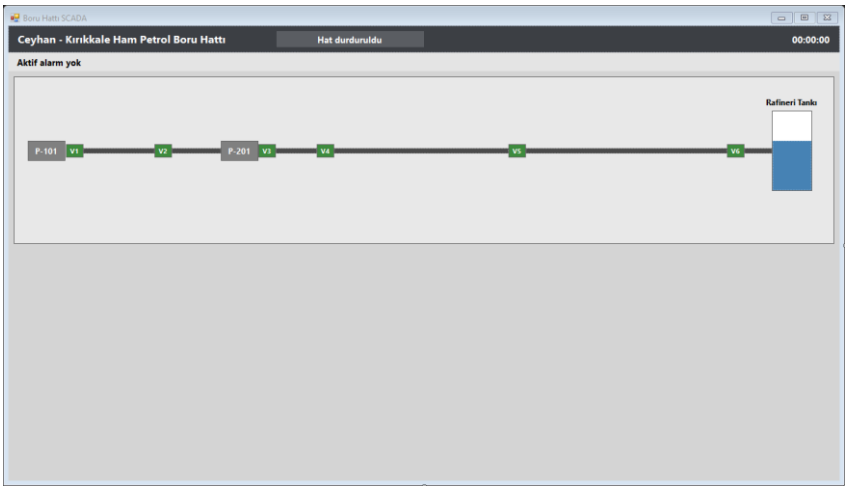
- **Pompa:** 56×30 piksel boyutunda, ortasında pompa adı yazan Label. Çalışırken yeşil, dururken gri, arızalıyken kırmızıdır.
- **Vana:** Boru üzerine yerleştirilmiş 26×20 piksel boyutunda Label. Açıkken yeşil, kapalıyken beyaz, haberleşme yokken gümüş renklidir.
- **Tank:** Beyaz bir Panel'in içine Dock = Bottom ayarlı mavi bir Panel konmuştur. Mavi panelin yüksekliği tank seviyesiyle orantılı olarak kodla değiştirilir.
- **İstasyon bilgileri:** Her vananın altında istasyon adını, kilometresini, basıncını ve debisini gösteren bir Label vardır.

4.2. Geliştirme Aşamaları

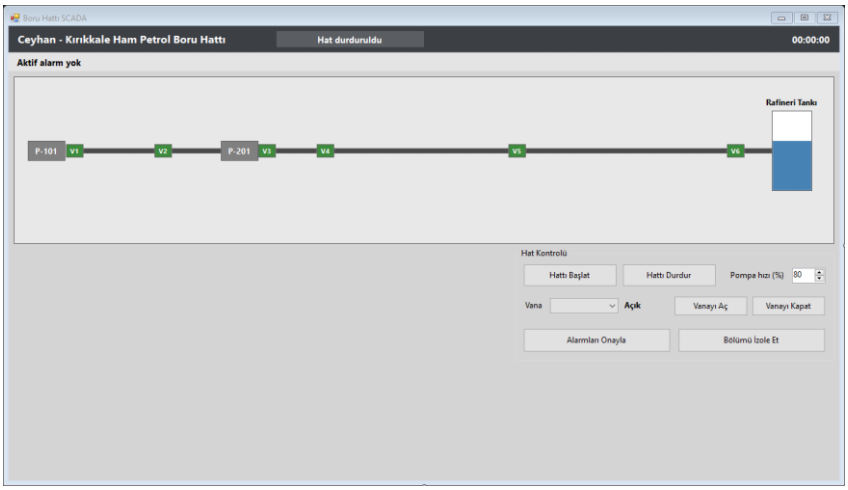
Arayüz adım adım oluşturulmuştur. Aşağıdaki şekiller Visual Studio tasarım ekranında formun her aşamadaki görünümünü göstermektedir.



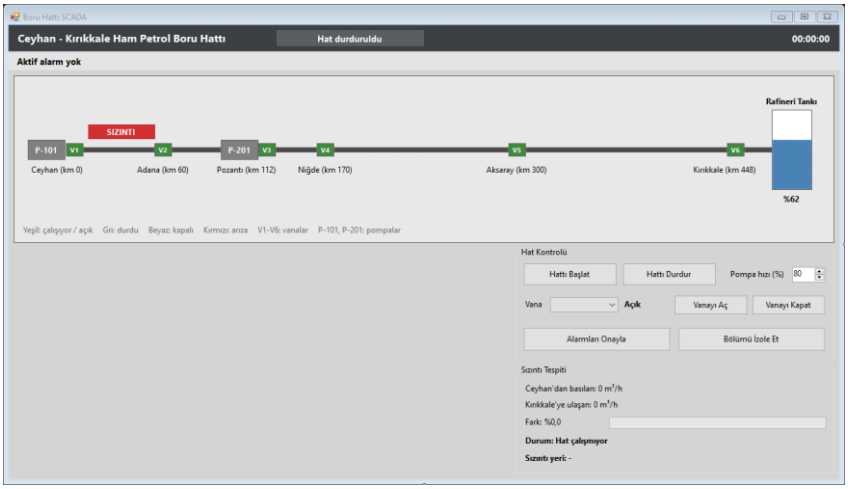
Şekil 6. Temel bileşenlerin eklenmesi: Üstte koyu renkli başlık paneli (hat adı, hat durumu ve saat) ile altında alarm bandı oluşturuldu.



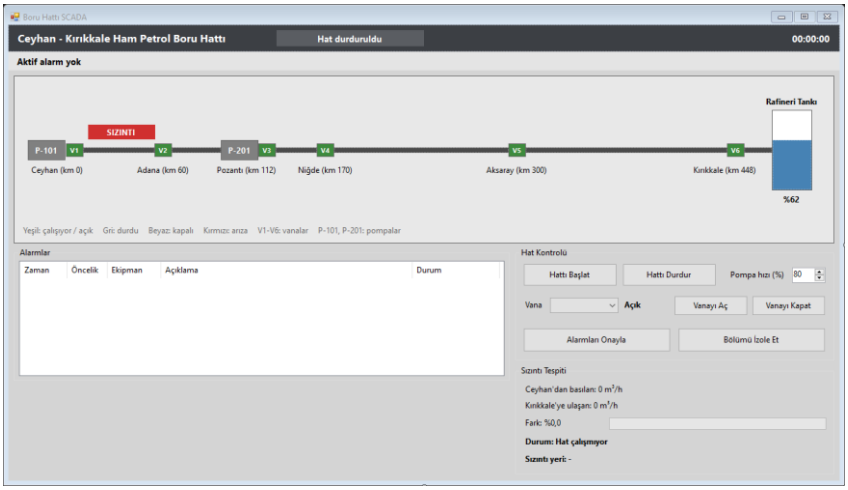
Şekil 7. Proses elemanlarının oluşturulması: Hat paneli eklendi. P-101 ve P-201 pompaları, V1–V6 vanaları, borular ve rafineri tankı Panel ve Label ile çizildi.



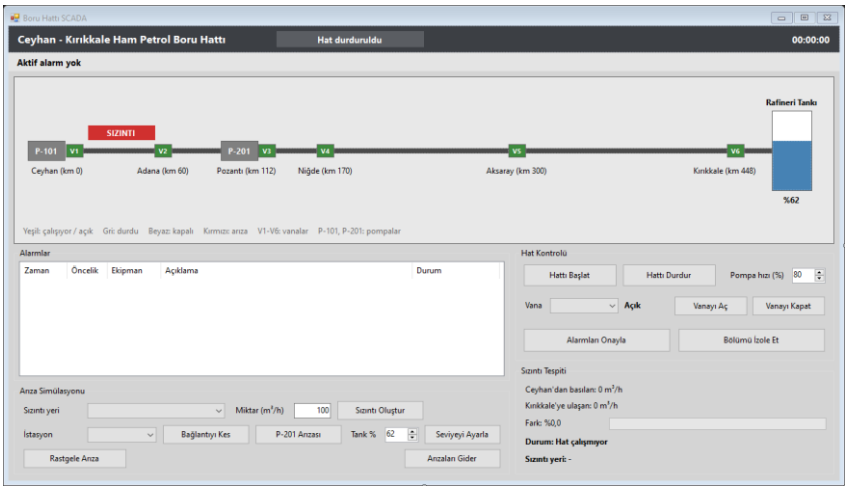
Şekil 8. Kontrol butonlarının eklenmesi: “Hat Kontrolü” GroupBox'ı içine başlatma/durdurma butonları, pompa hızı, vana seçimi ve vana butonları eklendi.



Şekil 9. Ölçüm göstergelerinin eklenmesi: İstasyonların altına basınç ve debi etiketleri, tank yüzde etiketi, renk açıklaması ve ProgressBar'lı “Sızıntı Tespiti” bölümü eklendi.



Şekil 10. Alarm alanının oluşturulması: Alarmlar GroupBox'ına beş sütunlu ListView eklendi.



Şekil 11. Tamamlanmış arayüz: Arıza Simülasyonu bölümü eklendi. Kırmızı "SIZINTI" etiketi tasarımda görünür, program çalışırken Visible = false yapılarak yalnızca sızıntı olduğunda gösterilir.

5. KODLAMA

Programın bütün kodu `AnaForm.cs` dosyasındadır. Tasarım ekranında eklenen bileşenlerin kodu Visual Studio tarafından `AnaForm.Designer.cs` dosyasına otomatik yazılır. İki dosya `partial` anahtar kelimesi sayesinde aynı sınıfın parçalarıdır. Aşağıda önemli kod parçaları verilmiş ve her birinin altında ne yaptığı açıklanmıştır.

5.1. Değişkenlerin Tanımlanması

```
string[] istasyonAdi = { "Ceyhan", "Adana", "Pozantı", "Niğde", "Aksaray", "Kırıkkale" };  // İstasyon
isimleri için dizi oluşturuldu
int[] istasyonKm = { 0, 60, 112, 170, 300, 448 }; // Çıkış noktasından itibaren uzaklıklar diziye
atandı, uyarı için


bool[] vanaAcik = { true, true, true, true, true, true };  // Vana açık kapalı bilgisi 1 bit olduğu
için bool
                                                                    // Başlangıç değerleri 0 olmasın diye
true girildi
bool[] baglantiVar = { true, true, true, true, true, true };  // İstasyon haberleşme bilgisi 1 bit
olduğu için bool
                                                                    // Başlangıç değerleri 0 olmasın diye
true girildi
bool[] vanaArizali = new bool[6];  // Vana arızalı bilgisi
double[] debi = new double[6];  // İstasyonda ölçülen debi (m³/h)
double[] basinc = new double[6];  // İstasyonda ölçülen basınç (bar)
```

Hattaki altı istasyonun bilgileri dizilerde tutulmuştur. Dizinin 0. elemanı Ceyhan, 5. elemanı Kırıkkale'dir. Örneğin `basinc[2]` Pozantı'nın basıncıdır. Vanaların açık/kapalı ve istasyonların haberleşme bilgisi iki durumlu olduğu için bool türündedir. Başlangıçta bütün vanalar açık ve bütün istasyonlarla haberleşme var olduğu için bu diziler `true` değerleriyle oluşturulmuştur. `new bool[6]` şeklinde oluşturulan diziler ise otomatik olarak false (veya 0) ile dolar.

```
bool p101Calisiyor = false;  // Ceyhan pompası çalışıyor mu?
bool p201Calisiyor = false;  // Pozantı pompası çalışıyor mu?
bool p201Arizali = false;  // Pozantı pompası arızalı mı?


double hatDebisi = 0;  // pompaların hatta bastığı petrol (m³/h)
double tankSeviye = 62;  // rafineri tankının doluluğu (%)


int sizintiBolum = -1;  // sızıntının olduğu bölüm. 0: V1-V2 arası, 1: V2-V3 arası ... -1: sızıntı yok
```

```
double sizintiMiktari = 0;  // sızan petrol (m³/h)

string hatDurumu = "Durduruldu";  // "Durduruldu", "Çalışıyor", "Sızıntı" veya "İzole"
double ortalamaFark = 0;  // giriş ile çıkış arasındaki farkın ortalaması (%)
int sinirAsimSuresi = 0;  // fark kaç saniyedir %1'in üstünde
int sizintiBaslangic = 0;  // sızıntı bu iki istasyonun arasında
int sizintiBitis = 0;  // (başlangıç ve bitiş istasyonunun sırası)
bool yenidenBaslatmaKilidi = false;  // izole edilen bölüm kontrol edilmeden hat başlatılmaz
bool basincKesmesi = false;  // basınç yükseldiği için pompaların durdurulma değişkeni
bool tankKesmesi = false;  // tank dolduğu için pompaların durdurulma değişkeni
int kapaliVanaSuresi = 0;  // pompa, kaç saniyedir kapalı vanaya karşı çalışıyor
```

Bu değişkenler sistemin o anki durumunu tutar. Butonlar bu değişkenleri değiştirir, Timer ise bu değişkenlere bakarak hesap yapar. `sizintiBolum` değişkeninde -1 değeri "sızıntı yok" anlamında kullanılmıştır. `hatDurumu` değişkeni hattın dört durumundan birini (Durduruldu, Çalışıyor, Sızıntı, İzole) yazı olarak tutar.

```
Random rastgele = new Random();  // rastgele sayı üretmek için ekledim
bool yanik = false;  // alarmların yanıp sönmesi için

// Kulanılacak renklerin renk kodunu atadım
Color yesil = Color.FromArgb(61, 139, 61);
Color kirmizi = Color.FromArgb(208, 48, 47);
Color turuncu = Color.FromArgb(232, 138, 0);
Color koyuGri = Color.FromArgb(74, 74, 74);
```

Random nesnesi bir kez oluşturulmuş ve bütün programda kullanılmıştır. Renkler RGB kodlarıyla bir kez tanımlanmış, böylece aynı renk kodu her yerde tekrar yazılmamıştır.

5.2. Formun Yüklenmesi

```
private void AnaForm_Load(object sender, EventArgs e) // Form ekranda ilk kez gösterilirken çalışır
{
    vanaEtiketleri = new Label[] { lblV1, lblV2, lblV3, lblV4, lblV5, lblV6 }; // Yukarıda oluşturulan
etikete atama yapıldı
    bilgiEtiketleri = new Label[] { lblCeyhan, lblAdana, lblPozanti, lblNigde, lblAksaray, lblKirikkale
};  // istasyon yazılarını sırayla diziye koydum
    borular = new Panel[] { pnlBoru1, pnlBoru2, pnlBoru3, pnlBoru4, pnlBoru5 };  // boru panellerini
sırayla diziye koydum


    // açılır listelerde ilk seçili gelecek satırlar program hata vermesin diye
    // SelectedIndex, listede kaçıncı satırın seçili olacağını belirler.
    cmbVana.SelectedIndex = 1;  // V2-Adana
    cmbIstasyon.SelectedIndex = 3;  // Niğde
    cmbSizintiBolum.SelectedIndex = 0;  // V1 - V2 (Ceyhan - Adana)


    tmrSimulasyon.Start();  // her saniye çalışan ana timer
    tmrYanipSonme.Start();  // alarmları yakıp söndüren timer
    EkraniGuncelle();  // program açılır açılmaz ekrandaki yazılar dolsun diye
}
```

Bu metot form açılırken bir kez çalışır. Etiketler ve paneller dizilere konduğu için ekran güncellenirken `vanaEtiketleri[i]` şeklinde döngü ile kullanılabilir. Dizilere atama bu metotta yapılmıştır, çünkü bileşenler `InitializeComponent()` çalışmadan önce oluşmamış olur. Son olarak iki Timer başlatılır.

5.3. Simülasyon Timer'ının Tick Olayı

```
// İşlemleri timera bağlı yaptırıyorum sürekli bir butona basılamaz
private void tmrSimulasyon_Tick(object sender, EventArgs e)
{
    DebiyiHesapla();  // önce istasyonlardaki debiyi buluyorum
    BasinciHesapla();  //Debiye bağlı olduğu için debinin altında
    TankiHesapla();  // Kırıkkale'ye gelen debiye göre tank dolar ya da boşalır
```

```
    SizintiKontrolEt();  // giriş ve çıkış debisini karşılaştırır
    AlarmlariKontrolEt();  // hesaplanan değerlere göre alarm listesini günceller
    EkraniGuncelle();  // en son bütün sonuçları ekrana yazar
}
```

Tick olayı her saniye çalışır ve hesaplamaları sırayla yapan metotları çağırır. Her işlem ayrı bir metoda yazıldığı için kod okunaklı olmuş ve aynı işlem tekrar yazılmamıştır. Basınç debiye bağlı olduğu için `BasinciHesapla()` metodu `DebiyiHesapla()` metodundan sonra çağrılır.

5.4. Debinin Hesaplanması ve Random ile Sensör Verisi Üretilmesi

```
void DebiyiHesapla()
{
    // Pompaların basması gereken debi. Pompa hızı %80 iken 1100 m³/h olur. ( İnternetten alınan
Ceyhan–Kırıkkale hattı bilgisi)
    double hedefDebi = 0;  // pompa çalışmıyorsa veya vana kapalıysa debi 0 olsun
    if (p101Calisiyor == true && KapaliVanaVarMi() == false)
    {
        hedefDebi = 1375 * (double)numHiz.Value / 100;  // 1375 = %100 hızdaki debi (1100 / 0,8).
numHiz decimal olduğu için double'a çevirdim

        // Pozantı pompası çalışmıyorsa petrol tek pompayla itilir, debi yarıya düşer
        if (p201Calisiyor == false)
        {
            hedefDebi = hedefDebi / 2;
        }
    }

    // Gerçekçi olması için debiyi her saniye hedefle arasındaki farkın %30u kadar arttırdım.
    hatDebisi = hatDebisi + (hedefDebi - hatDebisi) * 0.3;

    // Her istasyonun debisinin bulunması için döngü
    for (int i = 0; i < 6; i++)
    {
        // haberleşme yoksa yeni değer gelmez, ekranda eski değer kalır
        if (baglantiVar[i] == false)
        {
            continue; // göndüyü bu tür atla
        }

        double gercekDebi = hatDebisi;  // sızıntı yoksa her istasyondan hattın debisi geçer

        // Sızıntıdan sonraki istasyonlara, sızan miktar kadar az petrol ulaşır.
        // Örnek: sızıntı 1. bölümdeyse (V2-V3 arası) Pozantı ve sonrasında debi düşer.
        // sızıntıda bölüm varsa ve bu istasyon sızıntıdan sonraysa ve hatta petrol akıyorsa( akmıyorsa
debiyi etkilemesin diye)
        if (sizintiBolum != -1 && i > sizintiBolum && hatDebisi > 50)
        {
            gercekDebi = hatDebisi - sizintiMiktari;
        }

        // Sensör için hata payı eklendi
        // rastgele.NextDouble() ile 0-1 arası sayı oluşturuldu
        // - 0.5 ile aralık ortaya kaydırıldı. (-0.5 ile 0.5 arası rastgele sayı oluşturulmuş gibi
yapıldı)
        // %0,3 hata payı olması için 0.006 ile çarpıldı
        double hata = (rastgele.NextDouble() - 0.5) * 0.006;
        debi[i] = gercekDebi + gercekDebi * hata;  // gerçek debiye hata payını ekleyip sensörün
okuduğu değeri buldum
    }
}
```

Önce pompaların basması gereken **hedef debi** bulunur. Gerçek hatta pompa hızı %80 iken debi yaklaşık 1100 m³/h olduğu için %100 hızdaki debi 1100 / 0,8 = 1375 m³/h kabul edilmiştir. Hedef debi, `numHiz` kutusundaki yüzde ile çarpılarak bulunur. P-201 çalışmıyorsa

petrol tek pompa ile itildiği için debi yarıya düşer. Herhangi bir vana kapalıysa petrol akamaz ve hedef debi 0 olur.

Gerçek bir pompa hızını bir anda değiştiremez. Bu yüzden `hatDebisi` her saniye hedefe olan farkın %30'u kadar yaklaştırılmıştır. Örneğin debi 0 iken hedef 1100 ise ilk saniye 330, sonraki saniye 561, sonra 723 m³/h olur ve birkaç saniyede hedefe ulaşır.

Döngüde her istasyonun debisi bulunur. Haberleşmesi olmayan istasyonda `continue` ile döngü o tur için atlanır, böylece ekranda eski değer kalır. Sızıntı varsa sızıntıdan sonraki istasyonların debisi sızan miktar kadar azaltılır. Son olarak **Random ile sensör hata payı** eklenir: `rastgele.NextDouble()` 0 ile 1 arasında bir sayı üretir, 0,5 çıkarılınca sayı -0,5 ile +0,5 arasına kayar. Bu sayı 0,006 ile çarpılınca -0,003 ile +0,003 arası, yani **±%0,3** hata payı elde edilir. 1100 m³/h debide ekranda görülen değer yaklaşık 1097 ile 1103 arasında oynar.

5.5. Basıncın Hesaplanması

```
void BasinciHesapla()
{
    // Basınç pompanın çıkışında en yüksektir. Bir sonraki pompaya kadar
    // borudaki sürtünme yüzünden azalarak 5 bar'a iner
    // Debi 1100 m³/h iken pompa çıkışı 60 bar olur. Debi artarsa basınç da artar. (60 bar temsili
benim seçtiğim değer)
    double ceyhanCikis = 5 + 55 * hatDebisi / 1100; // (5) pompanın giriş basıncı, (55) pompanın normal
debide eklediği basınç
                                                    // (hatDebisi / 1100) o anki debinin normal debiye
oranı (hız %80 iken)

    double pozantiCikis = 5;  // P-201 çalışmıyorsa basıncı yükseltemez, 5 bar'da kalır
    if (p201Calisiyor == true)
    {
        pozantiCikis = 5 + 55 * hatDebisi / 1100;  // P-201 basıncı yeniden yükseltir
    }

    for (int i = 0; i < 6; i++)
    {
        if (baglantiVar[i] == false)
        {
            continue; // döngüyü bu tur için atla
        }

        // Bölge sınıflandırma
        if (istasyonKm[i] < 112)
        {
            // Ceyhan ile Pozantı arası (112 km)
            basinc[i] = ceyhanCikis - (ceyhanCikis - 5) * istasyonKm[i] / 112;
            // İstasyon       Km           Hesap            Basınç
            // Ceyhan       0       60 – 55 × 0 / 112     60,0
            // Adana       60       60 – 55 × 60 / 112    30,5
        }
        else
        {
            // Pozantı ile Kırıkkale arası (336 km)
            basinc[i] = pozantiCikis - (pozantiCikis - 5) * (istasyonKm[i] - 112) / 336;

            // İstasyon       Km           Hesap            Basınç
            // Pozantı    112       60 – 55 × 0 / 336     60,0
            // Niğde      170       60 – 55 × 58 / 336    50,5
            // Aksaray    300       60 – 55 × 188 / 336   29,2
            // Kırıkkale  448       60 – 55 × 336 / 336    5,0
        }
    }

    // Pompa çalışırken bir vana kapalıysa petrol gidecek yer bulamaz, hattın tamamında akış durur.
    // Pompalar petrolü itmeye devam ettiği için Ceyhan'dan kapalı vanaya kadar
```

```
    // bütün istasyonlarda (iki pompada da) basınç her saniye artar.
    int kapaliVana = IlkKapaliVana();
    if (p101Calisiyor == true && kapaliVana != -1)
    {
        kapaliVanaSuresi = kapaliVanaSuresi + 1;  // timer her saniye çalıştığı için saniye sayacı gibi
artar
        double yukselenBasinc = 60 + 6 * kapaliVanaSuresi;  // 66, 72, 78, 84

        // Ceyhan'dan (0) kapalı vananın olduğu istasyona kadar basınç yükselir
        for (int i = 0; i <= kapaliVana; i++)
        {
            basinc[i] = yukselenBasinc;
        }

        // Basınç 78 bar'ı geçerse boru zarar görmesin diye pompalar durdurulur
        if (yukselenBasinc > 78)
        {
            PompalariDurdur();
            basincKesmesi = true;  // alarm listesinde "BASINÇ" alarmı çıksın diye
        }
    }
    else
    {
        kapaliVanaSuresi = 0;  // vana açıldıysa veya pompa durduysa sayacı sıfırladım
    }
}
```

Basınç pompa çıkışında en yüksektir ve borudaki sürtünme yüzünden bir sonraki pompaya kadar azalarak 5 bar'a düşer. Normal debide (1100 m³/h) pompa çıkış basıncı temsili olarak 60 bar seçilmiştir. Formülde 5 bar pompanın giriş basıncı, 55 bar pompanın normal debide eklediği basınçtır. `hatDebisi / 1100` oranı sayesinde debi artınca basınç da artar. Pozantı'dan (km 112) önceki istasyonların basıncı P-101'e, sonraki istasyonların basıncı P-201'e göre hesaplanır. Kodun içindeki tablolarda her istasyon için hesap gösterilmiştir.

Metodun ikinci kısmı **kapalı vana durumunu** simüle eder. Pompa çalışırken bir vana kapanırsa petrol gidecek yer bulamaz. Pompalar itmeye devam ettiği için Ceyhan'dan kapalı vanaya kadar bütün istasyonlarda basınç her saniye 6 bar artar (66, 72, 78, 84 bar). Basınç 78 bar'ı geçince boruya zarar gelmemesi için pompalar otomatik olarak durdurulur ve `basincKesmesi` değişkeni true yapılarak alarm verilir.

```
// Ceyhan'dan başlayarak ilk kapalı vananın sırasını döndürür (V1 kapalıysa 0, V4 kapalıysa 3 ...).
// Kapalı vana yoksa -1 döner.
int IlkKapaliVana()
{
    for (int i = 0; i < 6; i++)
    {
        if (vanaAcik[i] == false)
        {
            return i;  // ilk kapalı vananın sırasını gönderir
        }
    }
    return -1;  // -1 diye bir sıra olmadığı için "kapalı vana yok" anlamında kullandım
}
```

Bu metot vanalara Ceyhan'dan başlayarak sırayla bakar ve ilk kapalı vananın sırasını döndürür. `return` komutu metodu hemen bitirdiği için ilk kapalı vana bulununca döngü durur. Kapalı vana yoksa -1 döndürülür.

5.6. Tank Seviyesinin Hesaplanması

```
void TankiHesapla()
{
    // Kırıkkale'ye gelen petrol tankı doldurur, rafineri saatte 1100 m³ çekerek boşaltır.
    // Tank 1500 m³. Debi saatlik olduğu için 3600'e bölünür (1 saniyelik değişim).
```

```
    double cekilen = 1100;
    if (tankSeviye < 5)
    {
        cekilen = 0;  // tank neredeyse boşsa rafineri çekmeyi durdurur
    }
    tankSeviye = tankSeviye + (debi[5] - cekilen) / 1500 * 100 / 3600;  // (giren - çıkan) / tank hacmi
* 100 ile yüzdeye çevirdim

    if (tankSeviye > 100) tankSeviye = 100;  // tank %100'ün üstüne çıkamaz
    if (tankSeviye < 0) tankSeviye = 0;  // tank %0'ın altına inemez

    // Tank taşmasın diye %95'te pompalar durdurulur
    if (tankSeviye > 95 && p101Calisiyor == true)
    {
        PompalariDurdur();
        tankKesmesi = true;  // alarm listesinde "TANK" alarmı çıksın diye
    }
}
```

Tanka Kırıkkale'ye ulaşan debi (`debi[5]`) kadar petrol girer, rafineri saatte 1100 m³ petrol çeker. Giren ile çıkan arasındaki fark tank hacmine (1500 m³) bölünüp 100 ile çarpılarak yüzdeye çevrilir. Debi saatlik olduğu için 3600'e bölünerek bir saniyelik değişim bulunur. Pompa çalışırken giren ve çıkan yaklaşık eşit olduğu için seviye çok yavaş değişir. Pompalar durunca tank yavaş yavaş boşalır. Seviye %95'i geçerse tank taşmasın diye pompalar durdurulur.

5.7. Sızıntı Tespiti

```
void SizintiKontrolEt()
{
    // Ceyhan veya Kırıkkale ile haberleşme yoksa giriş ve çıkış karşılaştırılamaz
    if (baglantiVar[0] == false || baglantiVar[5] == false)
    {
        ortalamaFark = 0;
        sinirAsimSuresi = 0;
        return;  // metodun geri kalanı çalışmasın diye çıkıyorum
    }

    // Ceyhan'dan basılan ile Kırıkkale'ye ulaşan arasındaki fark (yüzde olarak)
    double fark = 0;
    if (debi[0] > 100)  // hat çalışmıyorken (debi 0 iken) sıfıra bölme olmasın diye
    {
        fark = (debi[0] - debi[5]) / debi[0] * 100;
    }

    // Sensör hatası yüzünden fark biraz oynar. Bu yüzden ortalama alınır:
    // eski ortalamanın %80'i + yeni farkın %20'si
    ortalamaFark = ortalamaFark * 0.8 + fark * 0.2;

    // Fark 5 saniye boyunca %1'in üstünde kalırsa sızıntı alarmı verilir
    if (ortalamaFark > 1)
    {
        sinirAsimSuresi = sinirAsimSuresi + 1;  // her saniye 1 artar
    }
    else
    {
        sinirAsimSuresi = 0;  // fark düştüyse saymaya baştan başlar
    }

    if (sinirAsimSuresi >= 5 && hatDurumu == "Çalışıyor")
    {
        hatDurumu = "Sızıntı";
        SizintiYeriniBul();  // sızıntının hangi iki istasyon arasında olduğunu bulur
    }
}
```

Sızıntı, **debi dengesi** yöntemiyle tespit edilir: Ceyhan'dan basılan petrol ile Kırıkkale'ye ulaşan petrol arasındaki fark yüzde olarak hesaplanır. `debi[0] > 100` koşulu hat çalışmıyorken sıfıra bölme hatası olmaması için konmuştur. Sensör hata payı yüzünden fark her saniye biraz oynar. Bu yüzden tek bir ölçüme bakılmaz, ortalama alınır: yeni ortalamanın %80'i eski ortalamadan, %20'si yeni ölçümden gelir. Böylece tek bir hatalı ölçüm alarmı tetiklemez. Ortalama fark 5 saniye boyunca %1'in üstünde kalırsa hat durumu "Sızıntı" yapılır.

```
void SizintiYeriniBul()
{
    // İstasyonların debileri sırayla karşılaştırılır. Debinin düştüğü ilk iki istasyonun arasında
sızıntı vardır.
    // Örnek: Ceyhan 1100, Adana 1100, Pozantı 1000 ise sızıntı Adana ile Pozantı arasındadır.
    int onceki = 0;  // karşılaştırmaya Ceyhan'dan başlıyorum
    for (int i = 1; i < 6; i++)
    {
        if (baglantiVar[i] == false)
        {
            continue;  // haberleşme olmayan istasyon atlanır
        }

        if (debi[onceki] - debi[i] > 20)  // iki istasyon arasında 20 m³/h'ten fazla fark varsa (20
sensör hatası olmasın diye)
        {
            sizintiBaslangic = onceki;
            sizintiBitis = i;
            return;  // yeri bulduğum için döngüden çıkıyorum
        }
        onceki = i;  // fark yoksa bir sonraki istasyona geçiyorum
    }
}
```

Sızıntının yeri, yan yana istasyonların debileri karşılaştırılarak bulunur. Örneğin Ceyhan 1100, Adana 1000 m³/h ölçüyorsa aradaki 100 m³/h petrol bu iki istasyon arasında kaybolmuştur. 20 m³/h sınırı, sensör hata payının yanlış sonuç vermemesi için konmuştur. Haberleşmesi olmayan istasyon atlanır ve bir sonraki istasyonla karşılaştırma yapılır.

5.8. Ekranın Güncellenmesi ve Renklerin Değiştirilmesi

```
// Pompalar: çalışıyorsa yeşil, duruyorsa gri, arızalıysa kırmızı
if (p101Calisiyor == true) lblP101.BackColor = yesil;
else lblP101.BackColor = Color.Gray;

if (p201Arizali == true) lblP201.BackColor = kirmizi;
else if (p201Calisiyor == true) lblP201.BackColor = yesil;
else lblP201.BackColor = Color.Gray;
```

Bileşenlerin özellikleri kod içinden değiştirilerek cihazların durumu renkle gösterilir. `BackColor` özelliği Label'ın zemin rengidir. Tek satırlık if/else yapılarında süslü parantez kullanılmamıştır.

```
// Vana: açık yeşil, kapalı beyaz, haberleşme yoksa gri
if (baglantiVar[i] == false)
{
    vanaEtiketleri[i].BackColor = Color.Silver;
    vanaEtiketleri[i].ForeColor = Color.Black;
}
else if (vanaAcik[i] == true)
{
    vanaEtiketleri[i].BackColor = yesil;
    vanaEtiketleri[i].ForeColor = Color.White;
}
else
{
    vanaEtiketleri[i].BackColor = Color.White;
```

```
    vanaEtiketleri[i].ForeColor = Color.Black;  // beyaz zemin üstünde yazı siyah olmalı
}

// İstasyonun altındaki yazı (\n alt satıra geçer)
// ToString("0.0") virgülden sonra 1 basamak, ToString("0") hiç basamak göstermez
string yazi = istasyonAdi[i] + " (km " + istasyonKm[i] + ")\n"
            + "Basınç: " + basinc[i].ToString("0.0") + " bar\n"
            + "Debi: " + debi[i].ToString("0") + " m³/h";

if (baglantiVar[i] == false)
{
    yazi = yazi + "\nBağlantı yok!";  // haberleşme yoksa yazının altına ekledim
    bilgiEtiketleri[i].ForeColor = Color.Gray;  // değerler eski olduğu için gri
}
else
{
    bilgiEtiketleri[i].ForeColor = Color.Black;
}
bilgiEtiketleri[i].Text = yazi;  // hazırlanan yazıyı etikete yazdım
```

Altı istasyon tek döngü ile güncellenir. `ToString("0.0")` sayıyı virgülden sonra bir basamakla, `ToString("0")` ise tam sayı olarak yazar. `\n` karakteri yazıyı alt satıra geçirir. Haberleşmesi kopan istasyonun yazısı gri yapılır ve altına "Bağlantı yok!" eklenir.

```
// Tank: içteki mavi panelin yüksekliği doluluk kadar olur
pnlTankDolu.Height = (int)(pnlTank.Height * tankSeviye / 100);  // yükseklik tam sayı olmalı, (int) ile
çevirdim
lblTankSeviye.Text = "%" + tankSeviye.ToString("0");
```

```
double gosterilenFark = ortalamaFark;
if (gosterilenFark < 0) gosterilenFark = 0;  // sensör hatası yüzünden eksi çıkabilir
if (gosterilenFark > 10) gosterilenFark = 10;  // çubuk taşmasın diye en fazla %10
lblFark.Text = "Fark: %" + gosterilenFark.ToString("0.0");
prgFark.Value = (int)(gosterilenFark * 10);  // çubuk %10 farkta tamamen dolar (Maximum = 100)
```

Tank panelinin yüksekliği seviye ile orantılı olarak değiştirilir. Panel yüksekliği tam sayı olması gerektiği için `(int)` ile çevrilmiştir. ProgressBar'ın Value değeri 0–100 arasında olmalıdır. Fark önce 0–10 arasına sınırlanıp 10 ile çarpılır, böylece ProgressBar'a hiçbir zaman sınır dışı değer yazılmaz.

5.9. Kullanıcıdan Veri Alma ve Hata Kontrolü

```
private void btnSizintiOlustur_Click(object sender, EventArgs e)
{
    double miktar;  // kutuya yazılan sayı buraya gelecek

    // Kutuya yazılan değer sayı mı ve 50 ile 300 arasında mı?
    // TryParse: yazı sayıya çevrilebiliyorsa true döner ve sayıyı "miktar" değişkenine yazar
    if (double.TryParse(txtSizintiMiktar.Text, out miktar) == false || miktar < 50 || miktar > 300)
    {
        MessageBox.Show("Sızıntı miktarı 50 ile 300 arasında bir sayı olmalıdır.");
        txtSizintiMiktar.Focus();  // imleci tekrar kutuya götürür
        return;
    }

    sizintiBolum = cmbSizintiBolum.SelectedIndex;  // listeden seçilen bölüm
    sizintiMiktari = miktar;
    lblArizaBilgi.Text = "Sızıntı oluşturuldu: V" + (sizintiBolum + 1) + " - V" + (sizintiBolum + 2) +
" arası, " + miktar + " m³/h";

    if (p101Calisiyor == false)  // hat durmuşsa petrol akmadığı için sızıntı görünmez
    {
        lblArizaBilgi.Text = lblArizaBilgi.Text + " (hat çalışınca görülür)";
    }
}
```

TextBox'a yazılan değer yazı (string) türündedir ve sayıya çevrilmesi gerekir. `double.TryParse` metodu yazı sayıya çevrilebiliyorsa true döndürür ve sayıyı `out` ile `miktar` değişkenine yazar. Çevrilemiyorsa (örneğin "abc" yazıldıysa) false döndürür ve program hata vermez. Değer sayı değilse veya 50–300 aralığının dışındaysa kullanıcıya MessageBox ile uyarı gösterilir, imleç tekrar kutuya götürülür ve `return` ile metottan çıkılır.

```
private void btnTankAyarla_Click(object sender, EventArgs e)
{
    // Tank seviyesini elle değiştirip tank alarmlarını denemek için ekledim
    // NumericUpDown'da Minimum 0, Maximum 100 olduğu için yanlış değer girilemez
    // Value decimal olduğu için double'a çevirdim
    tankSeviye = (double)numTankSeviye.Value;
    lblArizaBilgi.Text = "Tank seviyesi %" + tankSeviye + " yapıldı";


    // %85 üstü yüksek, %20 altı düşük seviye alarmı verir. %95'i geçerse pompalar durur
    EkraniGuncelle();
}
```

NumericUpDown'ın Value özelliği decimal türünde olduğu için `(double)` ile çevrilmiştir. Minimum ve Maximum değerleri Properties'te 0 ve 100 olarak ayarlandığı için burada ayrıca hata kontrolü gerekmemiştir.

6. TIMER VE VERİ SİMÜLASYONU

6.1. Timer Neden Kullanıldı?

Gerçek SCADA sistemlerinde sahadan belirli aralıklarla yeni ölçüm gelir. Bunu taklit etmek için Timer kullanılmıştır: Interval süresi dolunca Tick olayı çalışır. Kod bir butona bağlansaydı değerler yalnızca butona basınca değişirdi, Timer sayesinde her saniye kendiliğinden güncellenir.

6.2. Interval Değerleri ve Tick Olayları

tmrSimulasyon (Interval = 1000 ms): Ana Timer'dır. Tick olayı 5.3 bölümünde verilmiştir. Debi değişimleri saatlik birimle (m³/h) hesaplandığı için bir saniyelik adım tank hesabında 3600'e bölünerek kullanılmıştır.

tmrYanipSonme (Interval = 500 ms): Alarm listesindeki yeni alarmları yakıp söndürür. Kodu 8.3 bölümünde verilmiştir.

tmrRastgeleAriza: Rastgele arıza butonuna basıldığında başlatılır. Interval değeri kodda rastgele seçilir ve Tick olayı bir kez çalıştıktan sonra Timer kendini durdurur:

```
private void btnRastgeleAriza_Click(object sender, EventArgs e)
{
    // Üçüncü timer: arıza 5 ile 20 saniye arasında rastgele bir anda oluşur
    tmrRastgeleAriza.Interval = rastgele.Next(5000, 20001);  // Next(5000, 20001): 5000 ile 20000 arası
(üst sınır dahil değil)
    tmrRastgeleAriza.Start();
    lblArizaBilgi.Text = "Rastgele arıza bekleniyor...";
}
```

```
private void tmrRastgeleAriza_Tick(object sender, EventArgs e)
{
    tmrRastgeleAriza.Stop();  // sadece bir kez çalışsın


    int arizaTuru = rastgele.Next(0, 4);  // 0, 1, 2 veya 3 sayısından biri
```

```
    if (arizaTuru == 0)
    {
        // Rastgele bir bölümde sızıntı
        sizintiBolum = rastgele.Next(0, 5);  // 0-4 arası, 5 bölümden biri
        sizintiMiktari = rastgele.Next(50, 151);  // 50-150 m³/h arası
    }
    else if (arizaTuru == 1)
    {
        // V2, V3, V4 veya V5 vanası kendiliğinden kapanır
        int i = rastgele.Next(1, 5);
        vanaAcik[i] = false;
        vanaArizali[i] = true;
    }
    else if (arizaTuru == 2)
    {
        // Ceyhan dışındaki bir istasyonla haberleşme kopar
        int i = rastgele.Next(1, 6);
        baglantiVar[i] = false;
    }
    else
    {
        // Pozantı pompası arızalanır
        p201Arizali = true;
        p201Calisiyor = false;
    }

    // Arızanın ne olduğu yazılmaz, operatör ekrana bakarak bulmalıdır
    lblArizaBilgi.Text = "Rastgele arıza oluştu!";
}
```

`rastgele.Next(5000, 20001)` 5000 ile 20000 arasında bir sayı üretir (üst sınır dahil değildir), yani arıza 5 ile 20 saniye arasında rastgele bir anda oluşur. `rastgele.Next(0, 4)` ile dört arıza türünden biri seçilir: sızıntı, vananın kendiliğinden kapanması, haberleşme kaybı veya P-201 arızası. Sızıntının bölümü ve miktarı, kapanacak vana ve haberleşmesi kopacak istasyon da rastgele seçilir. Ekranda yalnızca “Rastgele arıza oluştu!” yazar. Operatörün arızanın ne olduğunu ekrana bakarak bulması beklenir.

6.3. Değerlerin Sistem Durumuna Göre Değişmesi

Projede değerler yalnızca rastgele üretilmez, sistemin durumuna göre değişir. Aradaki ilişkiler Tablo 7'de özetlenmiştir.

Tablo 7. Sistem durumu ile ölçümler arasındaki ilişkiler

Durum	Sonuç
Hat başlatıldı	Debi birkaç saniyede 0'dan 1100 m³/h'e yükselir, pompa çıkış basınçları 60 bar'a çıkar.
Pompa hızı artırıldı/azaltıldı	Debi ve basınç hızla orantılı olarak değişir.
P-201 durdu / arızalandı	Debi yarıya düşer, Pozantı'dan sonraki basınç 5 bar'da kalır.
Bir vana kapatıldı	Akış durur, Ceyhan'dan kapalı vanaya kadar basınç her saniye 6 bar artar, 78 bar'ı geçince pompalar durur.
Sızıntı oluştu	Sızıntıdan sonraki istasyonların debisi düşer, giriş-çıkış farkı artar, ProgressBar dolar.
Haberleşme koptu	O istasyonun değerleri güncellenmez, gri görünür.
Pompalar durdu	Tanka petrol gelmez, rafineri çektiği için tank seviyesi yavaşça azalır.
Tank %95'i geçti	Tank taşmasın diye pompalar durdurulur.

7. SİSTEM KONTROLLERİNİN AÇIKLANMASI

7.1. Hattı Başlatma ve Durdurma

```
void HattiBaslat()
{
    // İzole edilen bölüm kontrol edilmeden hat çalıştırılmaz
    if (yenidenBaslatmaKilidi == true)
    {
        DialogResult cevap = MessageBox.Show("Sızıntılı bölüm kontrol edildi mi?", "Yeniden başlatma",
MessageBoxButtons.YesNo);
        if (cevap == DialogResult.No)
        {
            return;  // "Hayır" denirse hat başlatılmaz
        }
        yenidenBaslatmaKilidi = false;  // "Evet" denirse kilit kalkar
    }


    // Kapalı vana varsa hat başlatılmaz
    if (KapaliVanaVarMi() == true)
    {
        MessageBox.Show("Kapalı vana var. Önce vanaları açın.");
        return;
    }


    // Tank doluysa hat başlatılmaz, başlatılsa bile pompalar hemen durur
    if (tankSeviye > 95)
    {
        MessageBox.Show("Tank dolu (%" + tankSeviye.ToString("0") + "). Hat başlatılamaz.");
        return;
    }


    p101Calisiyor = true;
    if (p201Arizali == false)  // P-201 arızalıysa çalıştırılmaz
    {
        p201Calisiyor = true;
    }
    basincKesmesi = false;  // hat yeniden başladığı için eski kesme alarmları silinir
    tankKesmesi = false;
    ortalamaFark = 0;  // eski sızıntıdan kalan fark yeni çalışmada yanlış alarm vermesin diye
sıfırladım
    sinirAsimSuresi = 0;
    hatDurumu = "Çalışıyor";
    EkraniGuncelle();
}
```

Başlatma kodu iki yerden çağrıldığı için (Hattı Başlat butonu ve vana açıldıktan sonraki soru) ayrı bir metoda yazılmıştır. Hat başlatılmadan önce üç güvenlik kontrolü yapılır: izole edilmiş bölüm kontrol edildi mi, kapalı vana var mı ve tank dolu mu. Kontroller geçilirse pompaların değişkenleri true yapılır. P-201 arızalıysa çalıştırılmaz. Eski kesme alarmları ve sızıntı tespitinde kullanılan ortalama fark sıfırlanır.

```
// İki pompayı birden durdurur
void PompalariDurdur()
{
    p101Calisiyor = false;
    p201Calisiyor = false;
    if (hatDurumu == "Çalışıyor")  // sızıntı veya izole durumundaysa o yazı ekranda kalsın diye
    {
        hatDurumu = "Durduruldu";
    }
}
```

İki pompa birlikte durdurulur. Hat durumu yalnızca “Çalışıyor” ise “Durduruldu” yapılır. Böylece sızıntı veya izole durumunda pompalar durduğunda bu bilgi ekranda kalmaya devam eder.

7.2. Vana Açma ve Kapatma

```
private void btnVanaKapat_Click(object sender, EventArgs e)
{
    int i = cmbVana.SelectedIndex;  // listede seçili vananın sırası (0-5)

    if (baglantiVar[i] == false)
    {
        MessageBox.Show(istasyonAdi[i] + " istasyonu ile haberleşme yok. Vana kapatılamadı.");
        return;
    }

    // Pompa çalışırken vana kapatmak basıncı yükseltir, bu yüzden operatöre sorulur
    if (p101Calisiyor == true)
    {
        DialogResult cevap = MessageBox.Show("Pompa çalışırken vana kapatılırsa basınç yükselir. Yine
de kapatılsın mı?", "Vana", MessageBoxButtons.YesNo);
        if (cevap == DialogResult.No)
        {
            return;
        }
    }

    vanaAcik[i] = false;
    EkraniGuncelle();
}
```

`cmbVana.SelectedIndex` listede seçili satırın sırasını (0–5) verir ve bu sayı doğrudan dizi indisi olarak kullanılır. Haberleşmesi olmayan istasyondaki vanaya komut gönderilemez. Pompa çalışırken vana kapatılmak istenirse basınç yükseleceği için operatöre Evet/Hayır sorusu sorulur. Vana açma butonunda ise vana açıldıktan sonra bütün vanalar açık ve pompalar duruyorsa hattın yeniden başlatılması sorulur.

7.3. Sızıntılı Bölümün İzole Edilmesi

```
private void btnIzoleEt_Click(object sender, EventArgs e)
{
    if (hatDurumu != "Sızıntı")  // sızıntı yokken butona basılırsa uyarı verir
    {
        MessageBox.Show("İzole edilecek bir sızıntı yok.");
        return;
    }

    string vanalar = "V" + (sizintiBaslangic + 1) + " ve V" + (sizintiBitis + 1);  // dizi 0'dan
başladığı için +1 ekledim
    DialogResult cevap = MessageBox.Show("Pompalar durdurulacak, " + vanalar + " vanaları kapatılacak.
Onaylıyor musunuz?", "Bölüm izolasyonu", MessageBoxButtons.YesNo);
    if (cevap == DialogResult.No)
    {
        return;
    }

    // Pompalar durur, sızıntılı bölümün iki yanındaki vanalar kapanır
    PompalariDurdur();
    vanaAcik[sizintiBaslangic] = false;
    vanaAcik[sizintiBitis] = false;
    hatDurumu = "İzole";
    yenidenBaslatmaKilidi = true;  // tekrar başlatmadan önce "kontrol edildi mi?" diye sorulsun
    EkraniGuncelle();
}
```

Sızıntı yokken butona basılırsa uyarı verilir. Sızıntı varsa operatöre hangi vanaların kapatılacağı gösterilip onay istenir. Onay verilirse pompalar durdurulur, sızıntının iki yanındaki vanalar kapatılır ve yeniden başlatma kilidi açılır. Bu kilit, hat tekrar başlatılmak istendiğinde “Sızıntılı bölüm kontrol edildi mi?” sorusunun sorulmasını sağlar.

7.4. Arıza Simülasyonu Butonları

Tablo 8. Arıza simülasyonu butonları

Buton	Yaptığı iş
Sızıntı Oluştur	Seçilen bölümde, TextBox'a yazılan miktarda (50–300 m³/h) sızıntı oluşturur.
Bağlantıyı Kes	Seçilen istasyonla haberleşmeyi keser.
P-201 Arızası	Pozantı pompasını arızalı yapar.
Seviyeyi Ayarla	Tank seviyesini NumericUpDown'daki değere getirir (tank alarmlarını denemek için).
Rastgele Arıza	5–20 saniye içinde rastgele bir arıza oluşturur.
Arızaları Gider	Sızıntıyı, pompa arızasını ve haberleşme kayıplarını giderir. Kapalı vanaları operatörün açması gerekir.

```
private void btnArizalariGider_Click(object sender, EventArgs e)
{
    // Bütün arıza değişkenlerini başlangıç değerine döndürdüm
    tmrRastgeleAriza.Stop();  // bekleyen rastgele arıza varsa iptal olur
    sizintiBolum = -1;  // -1: sızıntı yok
    sizintiMiktari = 0;
    p201Arizali = false;
    if (p101Calisiyor == true)  // hat çalışıyorsa P-201 de tekrar çalışır
    {
        p201Calisiyor = true;
    }
    for (int i = 0; i < 6; i++)
    {
        baglantiVar[i] = true;  // bütün istasyonlarla haberleşme geri gelir
    }
    if (hatDurumu == "Sızıntı")
    {
        hatDurumu = "Çalışıyor";
    }
    ortalamaFark = 0;  // sızıntı giderildiği için fark ve sayaç sıfırlanır
    sinirAsimSuresi = 0;
    lblArizaBilgi.Text = "Arızalar giderildi (kapalı vanaları operatör açmalı)";
    EkraniGuncelle();
}
```

8. ALARM MEKANİZMASI

8.1. Alarm Koşulları ve Sınırları

Alarmlar önceliklerine göre üç gruba ayrılmıştır: 1 – kritik (kırmızı), 2 – önemli (turuncu), 3 – uyarı (sarı). Tablo 9'da bütün alarmlar verilmiştir.

Tablo 9. Alarm listesi

Ekipman	Öncelik	Koşul	Açıklama
LDS-01	1	Ortalama fark 5 s boyunca > %1	Sızıntı: (istasyon) ile (istasyon) arası
BASINÇ	1	Kapalı vana yüzünden basınç > 78 bar	Hat basıncı 78 bar'ı aştı, pompalar durduruldu
P-201	1	Pozantı pompası arızalı	Pozantı pompası arızalı, debi yarıya düştü
TANK	1	Tank seviyesi > %95	Tank doldu, pompalar durduruldu
TANK-H	2	Tank seviyesi > %85	Tank seviyesi yüksek

Ekipman	Öncelik	Koşul	Açıklama
TANK-L	2	Tank seviyesi < %20	Tank seviyesi düşük
V1 – V6	2	Vana kendiliğinden kapandı	Vx vanası kendiliğinden kapandı
İstasyon adı	3	İstasyonla haberleşme yok	(İstasyon) istasyonu ile haberleşme yok

Tank sınırları taşma ve boşalma riskine göre seçilmiştir: %85'te operatör uyarılır, %95'te ise sistem pompaları kendisi durdurur. 78 bar basınç sınırı, normal çalışma basıncı olan 60 bar'ın yaklaşık %30 üstü olarak temsili seçilmiştir.

8.2. Alarmların Kontrol Edilmesi

```
void AlarmlariKontrolEt()
{
    // AlarmDurumu(ekipman adı, öncelik, alarm var mı, açıklama)
    AlarmDurumu("LDS-01", "1", hatDurumu == "Sızıntı" || hatDurumu == "İzole", "Sızıntı: " +
SizintiYeri());
    AlarmDurumu("BASINÇ", "1", basincKesmesi, "Hat basıncı 78 bar'ı aştı, pompalar durduruldu");
    AlarmDurumu("P-201", "1", p201Arizali, "Pozantı pompası arızalı, debi yarıya düştü");
    AlarmDurumu("TANK", "1", tankKesmesi, "Tank doldu, pompalar durduruldu");
    AlarmDurumu("TANK-H", "2", tankSeviye > 85, "Tank seviyesi yüksek");
    AlarmDurumu("TANK-L", "2", tankSeviye < 20, "Tank seviyesi düşük");


    // Her istasyon için vana arızası ve haberleşme alarmı
    for (int i = 0; i < 6; i++)
    {
        AlarmDurumu("V" + (i + 1), "2", vanaArizali[i], "V" + (i + 1) + " vanası kendiliğinden
kapandı");
        AlarmDurumu(istasyonAdi[i], "3", baglantiVar[i] == false, istasyonAdi[i] + " istasyonu ile
haberleşme yok");
    }
}
```

```
// Alarm varsa listeye ekler, alarm geçtiyse listeden siler.
// Yeni eklenen alarmın durumu "Yeni" olur ve yanıp söner. Operatör onaylayınca "Görüldü" olur.
void AlarmDurumu(string ekipman, string oncelik, bool alarmVar, string aciklama)
{
    ListViewItem satir = lvAlarmlar.Items[ekipman];  // bu alarm listede var mı? (yoksa null döner)

    if (alarmVar == true && satir == null)  // alarm var ama listede yoksa ekle
    {
        satir = new ListViewItem(DateTime.Now.ToString("HH:mm:ss"));  // 1. sütun: zaman
        satir.Name = ekipman;  // alarmı sonra bulmak için ad veriyoruz
        satir.SubItems.Add(oncelik);  // 2. sütun
        satir.SubItems.Add(ekipman);  // 3. sütun
        satir.SubItems.Add(aciklama);  // 4. sütun
        satir.SubItems.Add("Yeni");  // 5. sütun
        lvAlarmlar.Items.Insert(0, satir);  // listenin en üstüne ekle
    }


    if (alarmVar == false && satir != null)  // alarm geçti ama listede duruyorsa sil
    {
        lvAlarmlar.Items.Remove(satir);
    }
}
```

Bütün alarmlar aynı metotla (`AlarmDurumu`) yönetilir. Böylece her alarm için ayrı if/else bloğu yazılmamıştır. Metodun üçüncü parametresi alarm koşuludur. Örneğin `tankSeviye > 85` ifadesi true veya false sonucunu verir ve bu sonuç doğrudan `alarmVar` parametresine gider. `lvAlarmlar.Items[ekipman]` ifadesi listede o isimde bir satır olup olmadığına bakar. Satır yoksa null döner. İki if koşulu şunu sağlar:

- Alarm var ve listede yoksa: zaman, öncelik, ekipman, açıklama ve “Yeni” durumu ile yeni satır oluşturulur ve listenin en üstüne eklenir.
- Alarm geçti ama listede duruyorsa: satır listeden silinir.

Alarm hem varken hem de listedeyken hiçbir şey yapılmaz. Bu sayede aynı alarm her saniye tekrar eklenmez.

8.3. Alarm Sırasında Arayüzdeki Değişiklikler

```
// İkinci timer: her yarım saniyede çalışır (Interval = 500 ms) ve yeni alarmları yakıp söndürür
private void tmrYanipSonme_Tick(object sender, EventArgs e)
{
    yanik = !yanik;  // true ise false, false ise true yapar

    foreach (ListViewItem satir in lvAlarmlar.Items)  // listedeki her alarm satırına sırayla bakar
    {
        if (satir.SubItems[4].Text == "Yeni" && yanik == false)
        {
            satir.BackColor = Color.White;  // sönük
        }
        else
        {
            satir.BackColor = AlarmRengi(satir.SubItems[1].Text);  // yanık
        }
    }

    // Üstteki alarm bandında listenin en üstündeki (en son gelen) alarm gösterilir
    if (lvAlarmlar.Items.Count == 0)
    {
        lblAlarm.Text = "Aktif alarm yok";
        pnlAlarm.BackColor = Color.FromArgb(228, 228, 228);  // alarm yoksa açık gri
    }
    else
    {
        ListViewItem ilk = lvAlarmlar.Items[0];  // listenin en üstündeki alarm
        lblAlarm.Text = ilk.Text + "     " + ilk.SubItems[2].Text + "     " + ilk.SubItems[3].Text;  //
zaman, ekipman, açıklama
        pnlAlarm.BackColor = ilk.BackColor;  // bant da alarmla birlikte yanıp söner
    }
}
```

```
// Alarmın önceliğine göre rengini döndürür
Color AlarmRengi(string oncelik)
{
    if (oncelik == "1") return Color.FromArgb(240, 120, 120);  // 1: kritik (kırmızı)
    if (oncelik == "2") return Color.FromArgb(250, 185, 90);  // 2: önemli (turuncu)
    return Color.FromArgb(245, 225, 110);  // 3: uyarı (sarı)
}
```

İkinci Timer her 500 ms'de `yanik` değişkenini tersine çevirir (`!` operatörü). Durumu “Yeni” olan satırlar bir beyaz, bir öncelik renginde boyanarak yanıp söner. Görülmüş alarmlar sürekli öncelik renginde kalır. Ekranın üstündeki alarm bandına listenin en üstündeki (en son gelen) alarmın zamanı, ekipmanı ve açıklaması yazılır, bandın rengi de o alarmla birlikte değişir. Alarm yoksa bant açık gri olur ve “Aktif alarm yok” yazar.

Alarm listesinin dışında ekranda başka değişiklikler de olur: sızıntıda üstteki hat durumu “SIZINTI ALARMI” yazısıyla kırmızı olur, sızıntılı boru parçası kırmızıya döner ve üzerinde “SIZINTI” etiketi görünür. Arızalı pompa kırmızı, haberleşmesi kopan istasyon gri gösterilir. Operatörün dikkatini çekmek için bazı durumlarda MessageBox ile soru veya uyarı penceresi açılır.

```
private void btnAlarmOnayla_Click(object sender, EventArgs e)
{
```

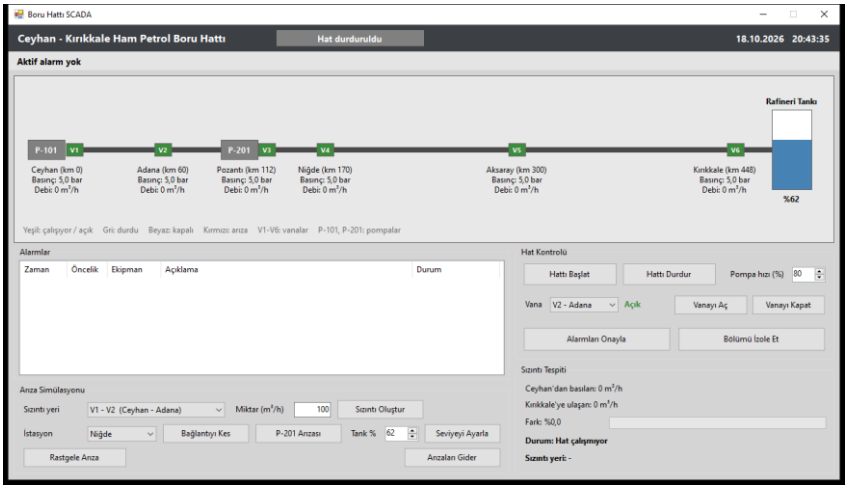
```
    // Bütün yeni alarmlar "Görüldü" olur, yanıp sönmeleri durur
    foreach (ListViewItem satir in lvAlarmlar.Items)
    {
        satir.SubItems[4].Text = "Görüldü";  // 5. sütun (Durum)
    }
}
```

Alarmları Onayla butonu listedeki bütün satırların 5. sütununu (indis 4) “Görüldü” yapar. Bu sütun artık “Yeni” olmadığı için satırlar yanıp sönmez. Alarm koşulu ortadan kalkınca satır listeden kendiliğinden silinir.

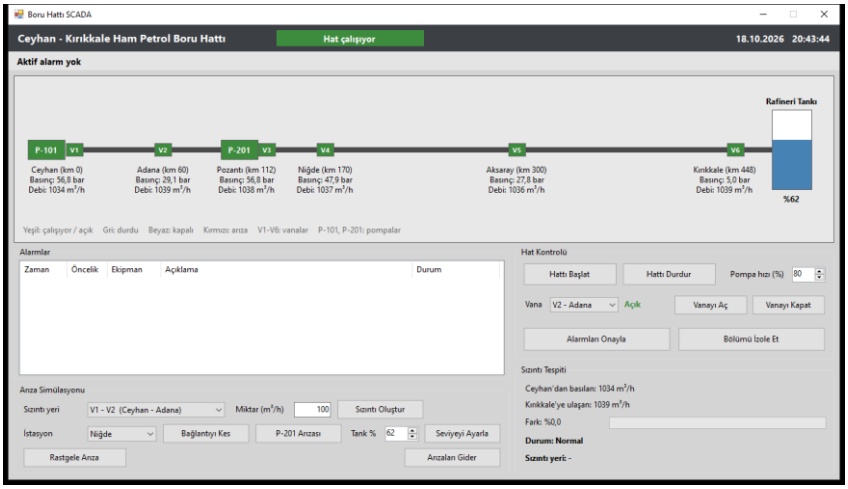
9. PROGRAMIN ÇALIŞTIRILMASI VE TEST EDİLMESİ

Program farklı durumlarda çalıştırılarak test edilmiştir. Her testte ekranın görüntüsü alınmış ve programın verdiği tepki aşağıda açıklanmıştır.

9.1. İlk Açılış ve Normal Çalışma

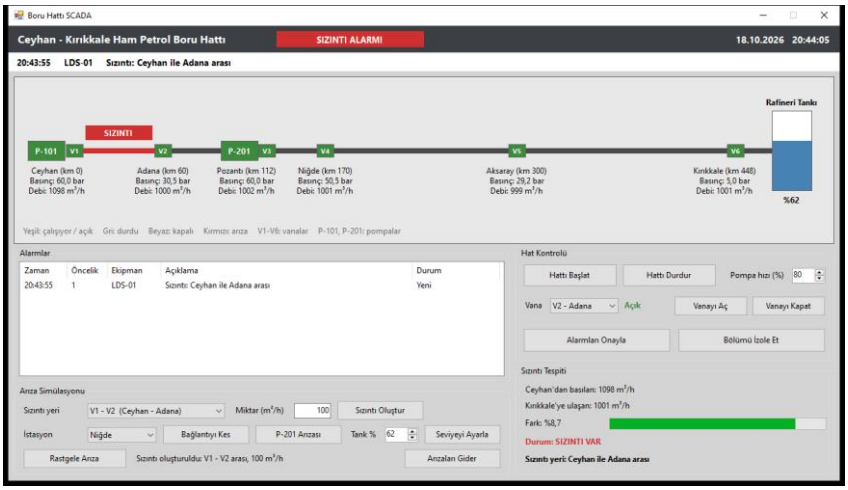


Şekil 12. Program ilk açıldığında: hat durmuş, pompalar gri, debiler 0 m³/h, bütün basınçlar 5 bar, tank %62, aktif alarm yok.

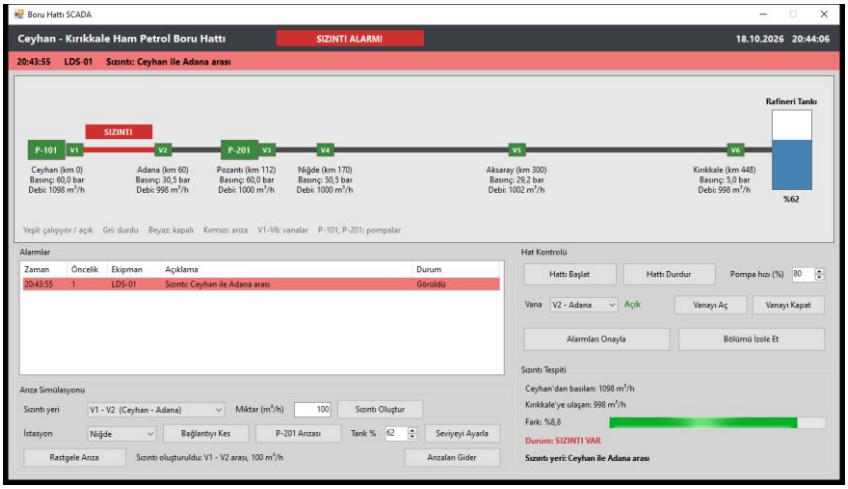


Şekil 13. “Hattı Başlat” butonuna basıldıktan birkaç saniye sonra: pompalar yeşil, debi hedefe doğru yükseliyor, hat durumu “Hat çalışıyor”. Normal çalışmanın görüntüsü 10. bölümde verilmiştir.

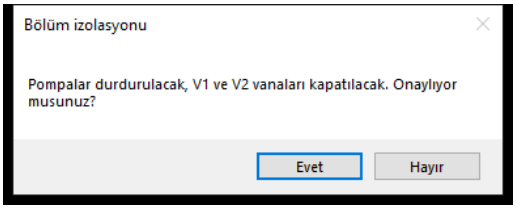
9.2. Sızıntı Alarmı, Alarm Onayı ve İzolasyon



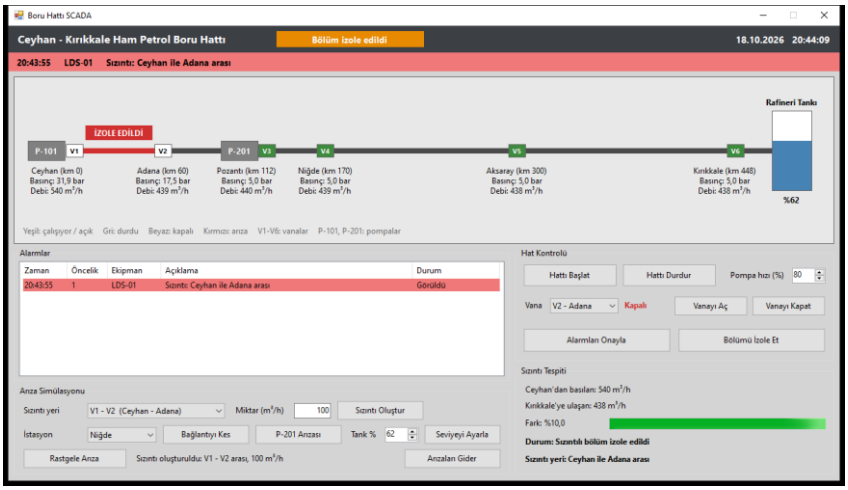
Şekil 14. V1–V2 arasında 100 m³/h sızıntı oluşturulduktan sonra: Adana ve sonrasında debi yaklaşık 1000 m³/h'e düştü, fark yaklaşık %9'a çıktı, ProgressBar doldu. Ceyhan–Adana arasındaki boru kırmızı, LDS-01 alarmı "Yeni" durumunda yanıp sönüyor.



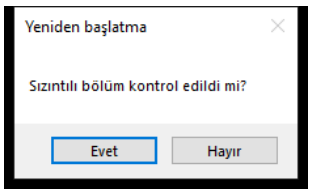
Şekil 15. "Alarmları Onayla" butonuna basıldıktan sonra alarmın durumu "Görüldü" oldu ve yanıp sönme durdu. Alarm bandı kırmızı kaldı.



Şekil 16. "Bölümü İzole Et" butonuna basıldığında operatörden onay istenmesi.

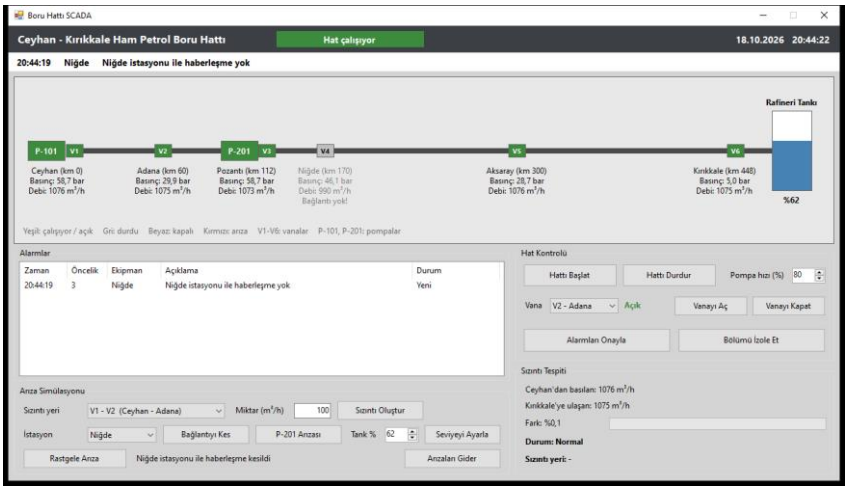


Şekil 17. Bölüm izole edildikten sonra: pompalar durdu, V1 ve V2 kapandı (beyaz), boru üzerinde "İZOLE EDİLDİ" yazıyor, hat durumu turuncu.

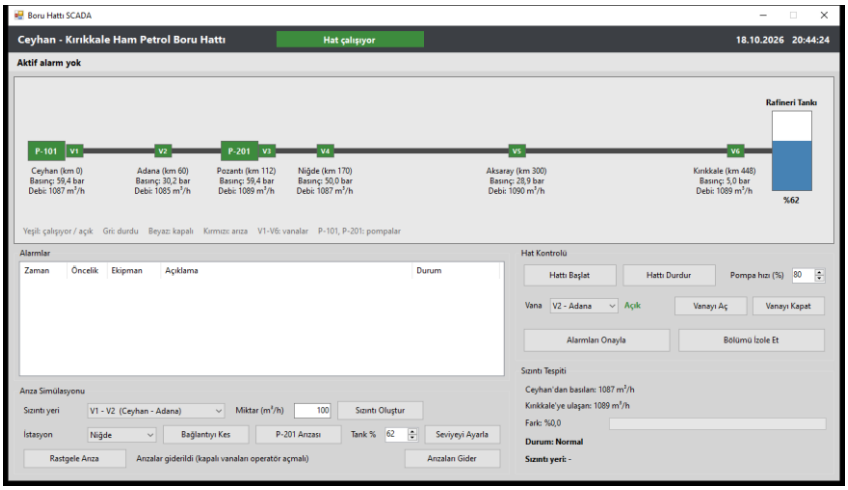


Şekil 18. Vanalar açılıp hat yeniden başlatılmak istendiğinde yeniden başlatma kilidinin sorduğu soru.

9.3. Haberleşme Kaybı ve Alarmın Ortadan Kalkması

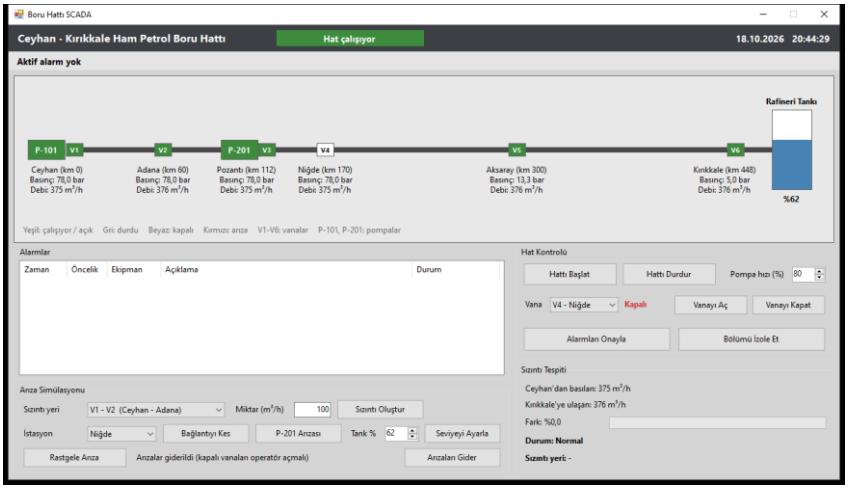


Şekil 19. Niğde istasyonu ile haberleşme kesildiğinde: V4 ve Niğde değerleri gri, "Bağlantı yok!" yazısı ve sarı (öncelik 3) alarm.

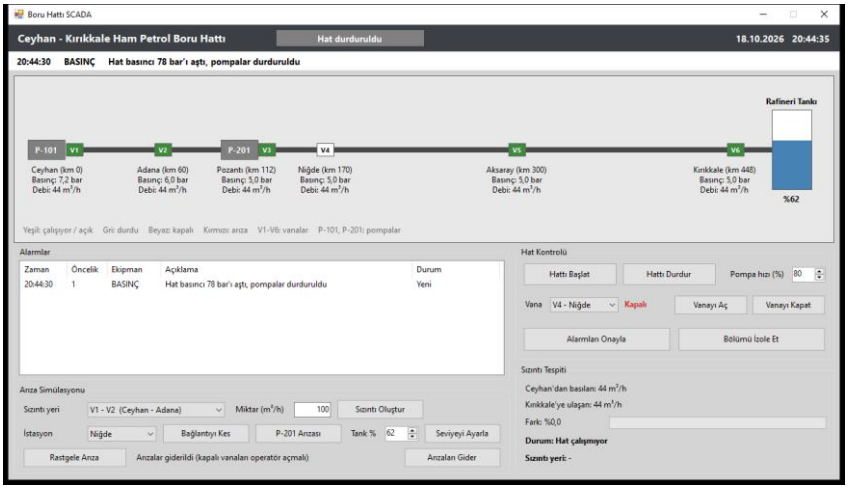


Şekil 20. "Arızaları Gider" butonuna basıldıktan sonra haberleşme geri geldi ve alarm listeden kendiliğinden silindi. Alarm bandında "Aktif alarm yok" yazıyor.

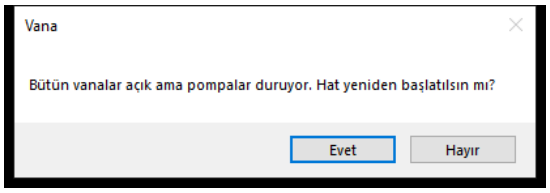
9.4. Vana Kapatılması ve Yüksek Basınç



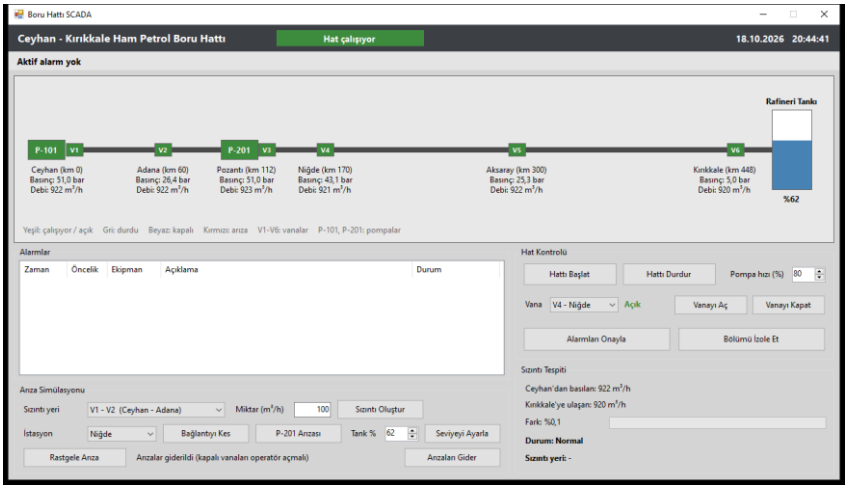
Şekil 21. Pompalar çalışırken V4 (Niğde) kapatıldı: akış durdu, Ceyhan'dan Niğde'ye kadar bütün istasyonlarda basınç 78 bar'a yükseldi.



Şekil 22. Basınç 78 bar'ı geçince pompalar otomatik olarak durduruldu ve kritik "BASINÇ" alarmı verildi.

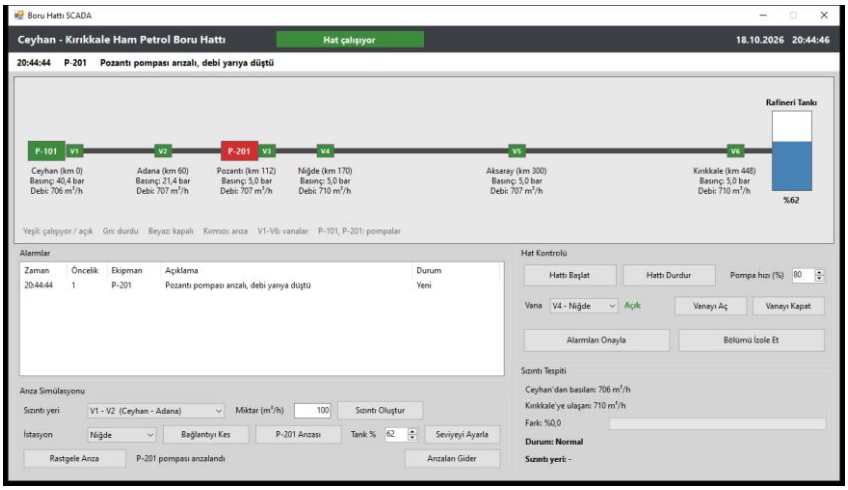


Şekil 23. V4 tekrar açıldığında bütün vanalar açık olduğu için hattın yeniden başlatılması soruldu.



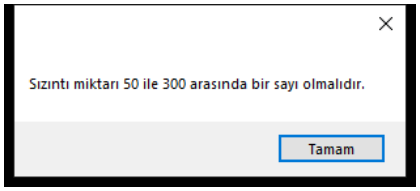
Şekil 24. "Evet" cevabından sonra hat yeniden çalıştı. Hat başladığı için BASINÇ alarmı ortadan kalktı.

9.5. Pompa Arızası



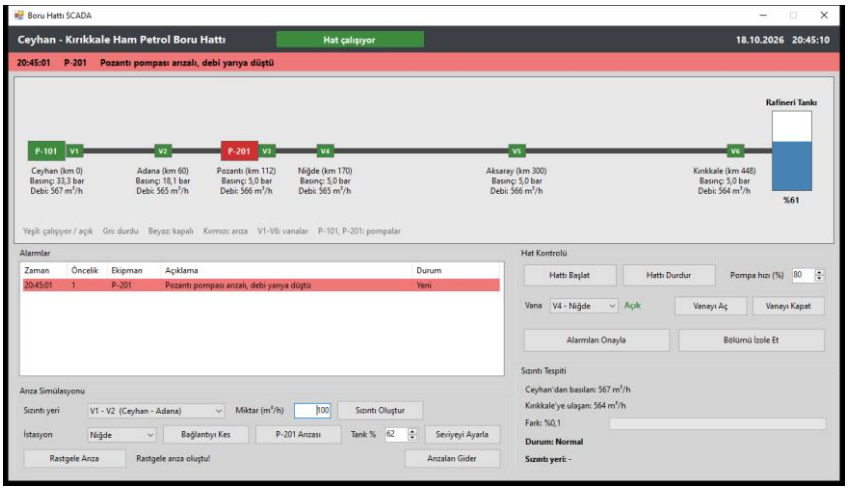
Şekil 25. P-201 arızası: Pozantı pompası kırmızı, debi yaklaşık yarıya düştü, Pozantı'dan sonraki basınçlar 5 bar'da kaldı, kritik alarm verildi.

9.6. Hatalı Kullanıcı Girişi



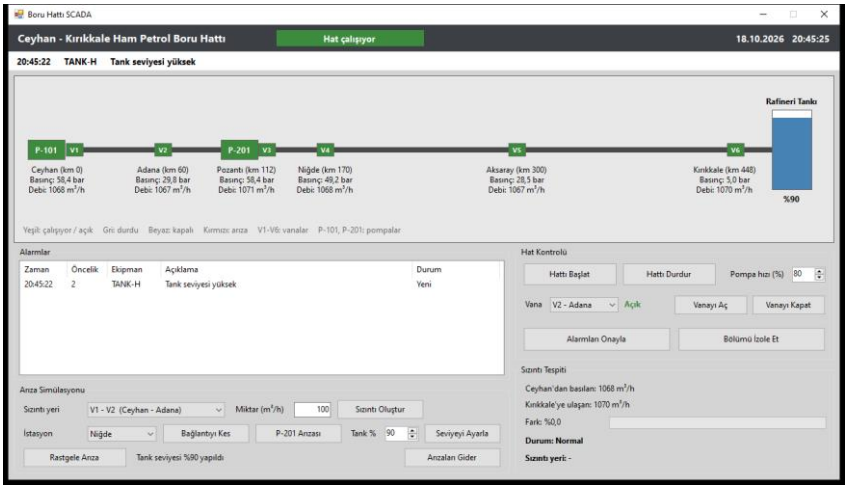
Şekil 26. Sızıntı miktarı kutusuna "abc" yazılıp "Sızıntı Oluştur" butonuna basıldığında verilen uyarı. Program hata vermeden çalışmaya devam etti.

9.7. Rastgele Arıza

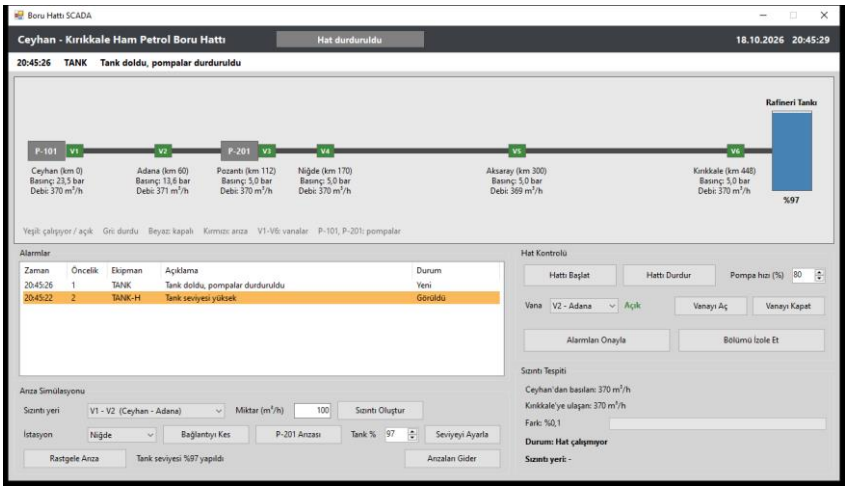


Şekil 27. "Rastgele Arıza" butonuna basıldıktan birkaç saniye sonra rastgele seçilen arıza oluştu. Bu denemede P-201 pompası arızalandı, operatör arızayı alarm listesinden ve kırmızı pompa sembolünden buldu.

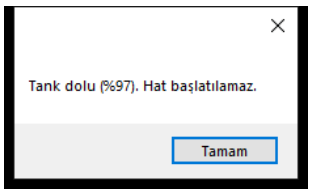
9.8. Tank Seviyesi Alarmları



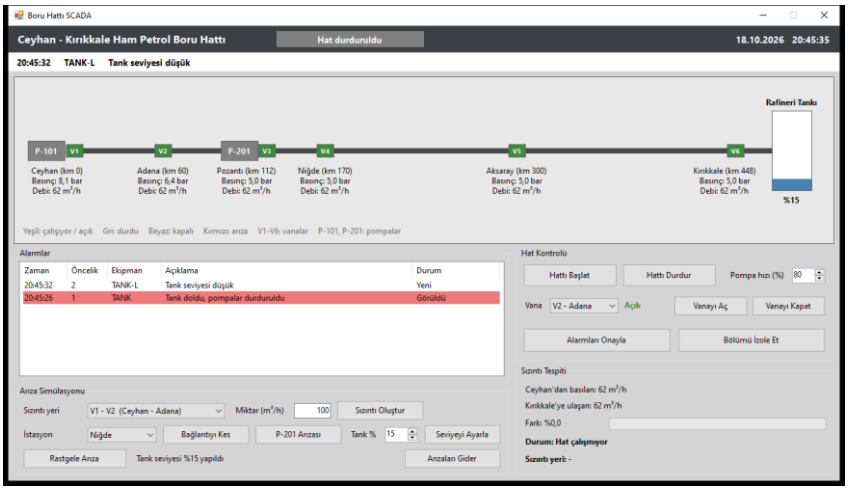
Şekil 28. Tank seviyesi %90 yapıldığında "Tank seviyesi yüksek" (TANK-H) alarmı turuncu renkte verildi.



Şekil 29. Tank seviyesi %97 yapıldığında pompalar otomatik durduruldu ve kritik "TANK" alarmı verildi.



Şekil 30. Tank doluyken hat başlatılmak istendiğinde verilen uyarı.



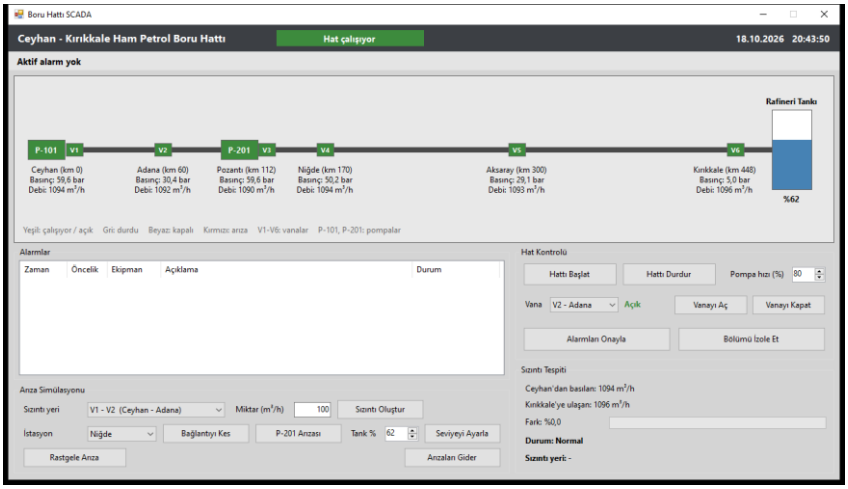
Şekil 31. Tank seviyesi %15 yapıldığında "Tank seviyesi düşük" (TANK-L) alarmı verildi. %85'in altına inildiği için TANK-H alarmı listeden silindi.

Tablo 10. Test sonuçlarının özeti

Test	Beklenen	Sonuç
Hattı başlatma	Debi ~1100 m³/h, basınç ~60 bar	Başarılı
Sızıntı (V1–V2, 100 m³/h)	LDS-01 alarmı, yer: Ceyhan–Adana	Başarılı
Alarm onaylama	Durum "Görüldü", yanıp sönme durur	Başarılı
Bölüm izolasyonu	Pompalar durur, V1 ve V2 kapanır	Başarılı
Haberleşme kaybı	Değerler gri, sarı alarm	Başarılı
Vana kapatma	Basınç artar, 78 bar'da pompalar durur	Başarılı
P-201 arızası	Debi yarıya düşer, kritik alarm	Başarılı

Test	Beklenen	Sonuç
Hatalı giriş (“abc”)	Uyarı mesajı, program çökmez	Başarılı
Rastgele arıza	5–20 s içinde rastgele arıza	Başarılı
Tank %90 / %97 / %15	TANK-H / TANK + pompa durması / TANK-L	Başarılı

10. PROGRAMIN SON HÂLİ



Şekil 32. SCADA uygulamasının son hâli (normal çalışma): debi yaklaşık 1100 m³/h, pompa çıkışlarında yaklaşık 60 bar, Kırıkkale'de 5 bar, fark %0,1.

Ana ekran beş bölümden oluşur:

- **Başlık ve alarm bandı (üst):** Hattın adı, hattın genel durumu (renkli kutu), tarih-saat ve en son gelen alarm.
- **Hat şeması (orta):** P-101 ve P-201 pompaları, V1–V6 vanaları, borular, rafineri tankı ve her istasyonun basınç (bar) ve debi (m³/h) değerleri. Görüntülenen sensörler: 6 istasyonda debi ve basınç, tankta seviye. Altta renklerin anlamı yazılıdır.
- **Alarmlar (sol orta):** Zaman, öncelik, ekipman, açıklama ve durum sütunlarından oluşan alarm listesi.
- **Hat Kontrolü (sağ üst):** Hattı başlatma/durdurma butonları, pompa hızı girişi, vana seçimi, vana açma/kapatma, alarm onaylama ve bölüm izolasyonu butonları.
- **Sızıntı Tespiti (sağ alt):** Ceyhan'dan basılan ve Kırıkkale'ye ulaşan debi, aradaki farkı gösteren ProgressBar, sızıntı durumu ve sızıntının yeri.
- **Arıza Simülasyonu (sol alt):** Sistemi denemek için sızıntı, haberleşme kaybı, pompa arızası, tank seviyesi ayarı ve rastgele arıza oluşturma alanı. Gerçek bir SCADA ekranında bu bölüm bulunmaz, yalnızca simülasyon amacıyla eklenmiştir.

11. SONUÇ

11.1. Gerçekleştirilenler

Bu projede Ceyhan – Kırıkkale ham petrol boru hattını örnek alan bir SCADA operatör ekranı C# Windows Forms ile geliştirilmiştir. Program iki pompa istasyonunu, altı vana istasyonunu ve rafineri tankını izlemekte; debi, basınç ve tank seviyesini her saniye hesaplayıp

ekranda göstermektedir. Debi dengesi yöntemiyle sızıntı tespit edilmekte ve sızıntının hangi iki istasyon arasında olduğu bulunmaktadır. Kapalı vana yüzünden yükselen basınçta ve dolan tankta pompalar otomatik olarak durdurulmakta, bütün anormal durumlar öncelikli bir alarm listesinde gösterilmektedir. Operatör hattı başlatıp durdurabilmekte, pompa hızını değiştirebilmekte, vanaları açıp kapatabilmekte, alarmları onaylayabilmekte ve sızıntılı bölümü izole edebilmektedir.

11.2. Kullanılan C# Konuları

- Değişkenler ve veri türleri (int, double, bool, string, Color), diziler
- if / else if / else yapıları, mantıksal operatörler (&&, ||, !)
- for ve foreach döngüleri, continue ve return
- Değer döndüren ve döndürmeyen metotlar, parametreli metotlar
- Olay tabanlı programlama: Load, Click, Tick ve SelectedIndexChanged olayları
- Timer ile zamanlama, Random sınıfı ile rastgele sayı üretimi
- Kullanıcıdan veri alma (TextBox, NumericUpDown, ComboBox) ve TryParse ile hata kontrolü
- Bileşen özelliklerinin (Text, BackColor, ForeColor, Height, Left, Visible, Value) koddan değiştirilmesi
- MessageBox ile uyarı ve Evet/Hayır soruları (DialogResult)

11.3. Karşılaşılan Problemler ve Çözümleri

- **Sensör hatası yüzünden yanlış alarm riski:** Debi değerleri her saniye oynadığı için tek ölçüme bakmak yanlış alarm verebiliyordu. Farkın ortalaması alınarak ve sınırın 5 saniye aşılması şartı konarak çözüldü.
- **Yeniden başlatmada tekrar gelen sızıntı alarmı:** Bölüm izole edilip hat yeniden başlatıldığında eski ortalama fark sıfırlanmadığı için sızıntı alarmı hemen tekrar geliyordu. Hat başlatılırken ve arızalar giderilirken ortalama fark ile sayaç sıfırlanarak çözüldü.
- **Debinin bir anda değişmesi:** Hat başlatıldığında debi bir anda 1100 m³/h oluyordu ve gerçekçi değildi. Debinin her saniye hedefe olan farkın %30'u kadar artması sağlandı.
- **Vana kapanınca basıncın sadece bir istasyonda artması:** İlk hâlinde basınç yalnızca bir pompada artıyordu. Akış bütün hatta durduğu için basıncın Ceyhan'dan kapalı vanaya kadar bütün istasyonlarda artması sağlandı.
- **Vana kapatılıp açıldığında hattın çalışmaması:** Yüksek basınç yüzünden pompalar durduğu için vana açılınca debi gelmiyordu. Bütün vanalar açıldığında hattın yeniden başlatılması soruldu.
- **Okunmayan vana yazısı:** Kapalı vananın zemini beyaz olduğu için beyaz yazı görünmüyordu. Kapalı vanalarda yazı rengi siyah yapıldı.
- **Altı istasyonun tek tek güncellenmesi:** Her istasyon için ayrı kod yazmak yerine Label ve Panel'ler dizilere konup döngü ile güncellendi.

11.4. Kazanımlar

Bu proje ile bir endüstriyel sistemden gelen verilerin ekranda nasıl gösterileceği, Timer ile nasıl güncelleneceği ve cihaz durumlarının renklerle nasıl görselleştirileceği uygulamalı olarak öğrenilmiştir. Hesaplama kodunun ekran kodundan ayrılmasının, tekrar eden işlemlerin metotlara bölünmesinin ve dizilerle döngü kullanmanın kodu ne kadar kısalttığı görülmüştür. Ayrıca gerçek bir boru hattının işletilmesi, sızıntı tespit yöntemleri, vana izolasyonu ve alarm önceliklendirme gibi SCADA kavramları hakkında bilgi edinilmiştir.